COMPLETE FRONTEND BLUEPRINT:

Fundraising Platform — Angular 22

Complete frontend notes • architecture • signals • RxJS • forms • MSAL • testing • accessibility • Azure deployment

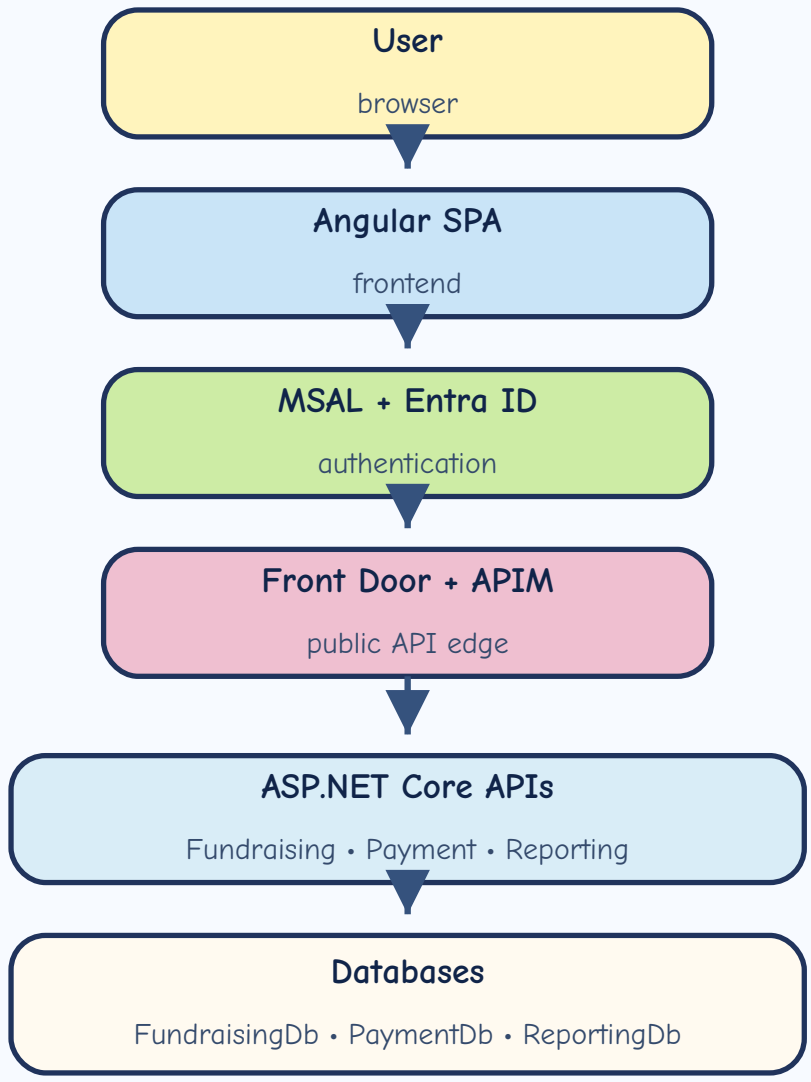
INDEX — Main Topics & Important Subtopics

★ Study flow: Angular foundations → feature architecture → reactive state → HTTP → authentication → forms → business screens → accessibility/performance → testing → deployment → complete project architecture.

- Purpose & User-to-API Architecture
- Angular 22 Baseline & Standalone Architecture
- Feature Architecture: Core / Shared / Features
- Application Shell, Routing & Lazy Loading
- Components, Inputs & Outputs
- Signals, computed(), effect() & RxJS
- State Management, Stores & NgRx Decision
- HTTP Clients, Interceptors & API Configuration
- Microsoft Entra ID, MSAL, Tokens & Authorization
- Signal Forms, Reactive Forms & Validation
- Tables, Pagination, Filtering, Sorting & Search
- Dashboard & Reporting API Design
- Constituents, Campaigns, Funds & Gifts
- Tasks, Payments, Refunds & Settlements
- Reporting, Charts & Administration
- Design System, Angular Material & Accessibility
- Zoneless, OnPush & Performance
- Browser Storage, XSS, XSRF & CORS
- Frontend Patterns: Adapter, Facade, DI & Directives
- Resource APIs & Modern Route Data Loading
- Testing: Vitest, HTTP, Guards & E2E
- Logging, Telemetry & Failure UX
- Route Map, Titles, Breadcrumbs & Navigation Guards
- Frontend Boundaries: APIs, Service Bus & Key Vault
- Azure Hosting, SPA Fallback & GitHub CI/CD
- Testing Pyramid & Topic Map
- Authentication, Gift, Payment, Dashboard & Error Flows
- State & Performance Architecture
- Recommended Build Sequence
- Angular Completion Checklist
- Final Architecture, Memory Rules & End-to-End Journey

DIAGRAM

Angular 22 - User to Data Request Path



The browser authenticates through MSAL/Entra, reaches APIs through the Azure edge, and never connects directly to databases.

Purpose of This Document

This document builds the frontend for the same platform we already designed:

EXAMPLE / FLOW

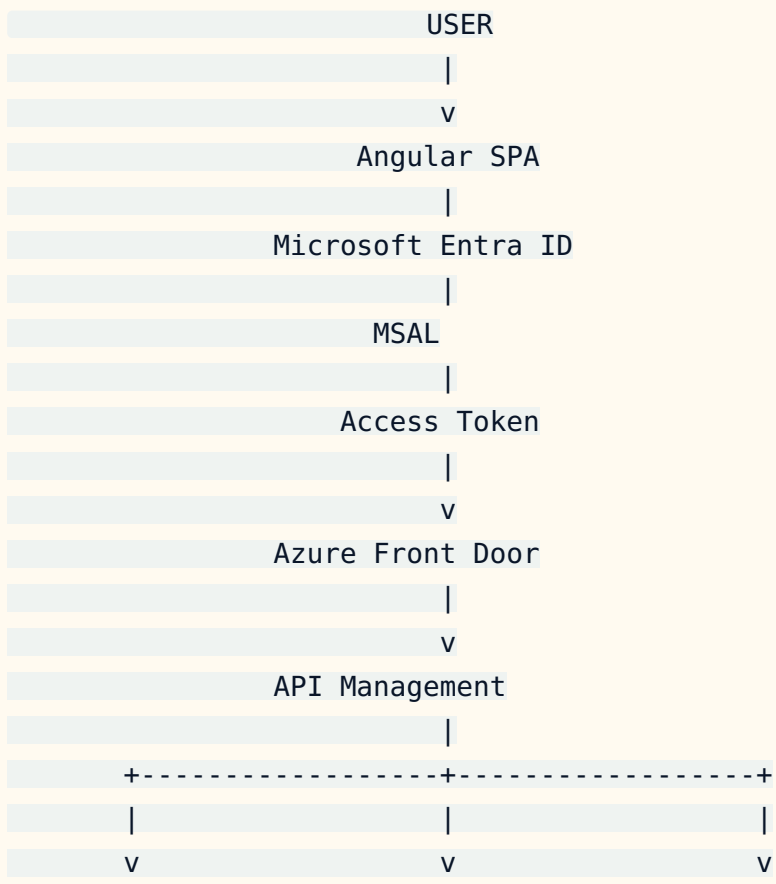
```
FundraisingDb
      |
Fundraising API

PaymentDb
      |
Payment API

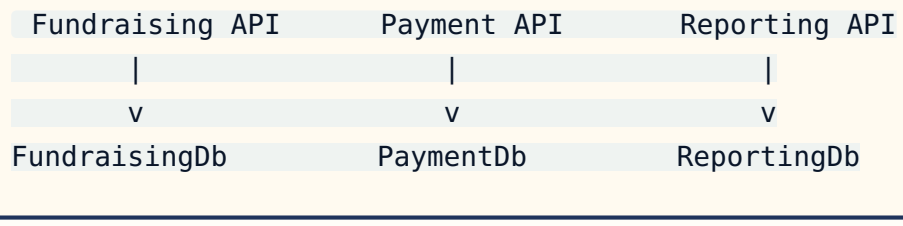
ReportingDb
      |
Reporting API
```

The Angular application becomes the user-facing system:

EXAMPLE / FLOW



EXAMPLE / FLOW



The application must support:

EXAMPLE / FLOW

Authentication
Authorization
Tenant-aware user experience
Dashboard
Constituents
Addresses
Campaigns
Funds
Gifts
Gift allocations
Interactions
Tasks
Payments
Payment attempts
Refunds
Settlements
Reporting
Administration

It must also teach:

EXAMPLE / FLOW

```
Angular architecture
Standalone components
TypeScript
Components
Templates
Bindings
Signals
Computed signals
Effects
RxJS
Services
Dependency Injection
Routing
Lazy loading
Route guards
HTTP
Interceptors
Forms
Validation
State management
```

EXAMPLE / FLOW

```
Tables
Pagination
Filtering
Sorting
Search
Dialogs
Reusable components
Error handling
Loading states
Authentication
MSAL
Role-based UI
Authorization
Accessibility
Performance
Testing
Vitest
HTTP tests
Component tests
E2E concepts
```

EXAMPLE / FLOW

```
CI/CD
Azure deployment
```

1. Angular Technology Baseline

Use:

EXAMPLE / FLOW

```
Angular 22.x
TypeScript
Angular CLI
Standalone architecture
Signals
RxJS
Signal Forms
HttpClient
Angular Router
MSAL Angular / MSAL Browser
SCSS
Vitest
```

Angular 22 is the active supported major as of this project baseline. New Angular projects are zoneless by default, and Angular 22 uses `OnPush` as the default change detection strategy. Signal Forms are stable in Angular 22.

2. Do We Need NgModules?

For a new Angular application:

EXAMPLE / FLOW

```
No traditional AppModule architecture
```

should be necessary.

Use:

EXAMPLE / FLOW

```
Standalone components
Standalone directives
Standalone pipes
provideRouter()
provideHttpClient()
bootstrapApplication()
```

Angular supports standalone-first architecture and provides migrations away from traditional module-heavy application organization.

That does **not** mean:

EXAMPLE / FLOW

```
NgModule is useless
```

because libraries and older enterprise applications still use it.

You should understand NgModules because you will encounter them professionally.

For this project, however:

EXAMPLE / FLOW

```
modern standalone architecture
```

is the default.

3. Create the Project

Conceptually:

CLI

```
ng new fundraising-portal \
  --routing \
  --style=scss \
  --strict
```

Then:

EXAMPLE / FLOW

```
cd fundraising-portal
```

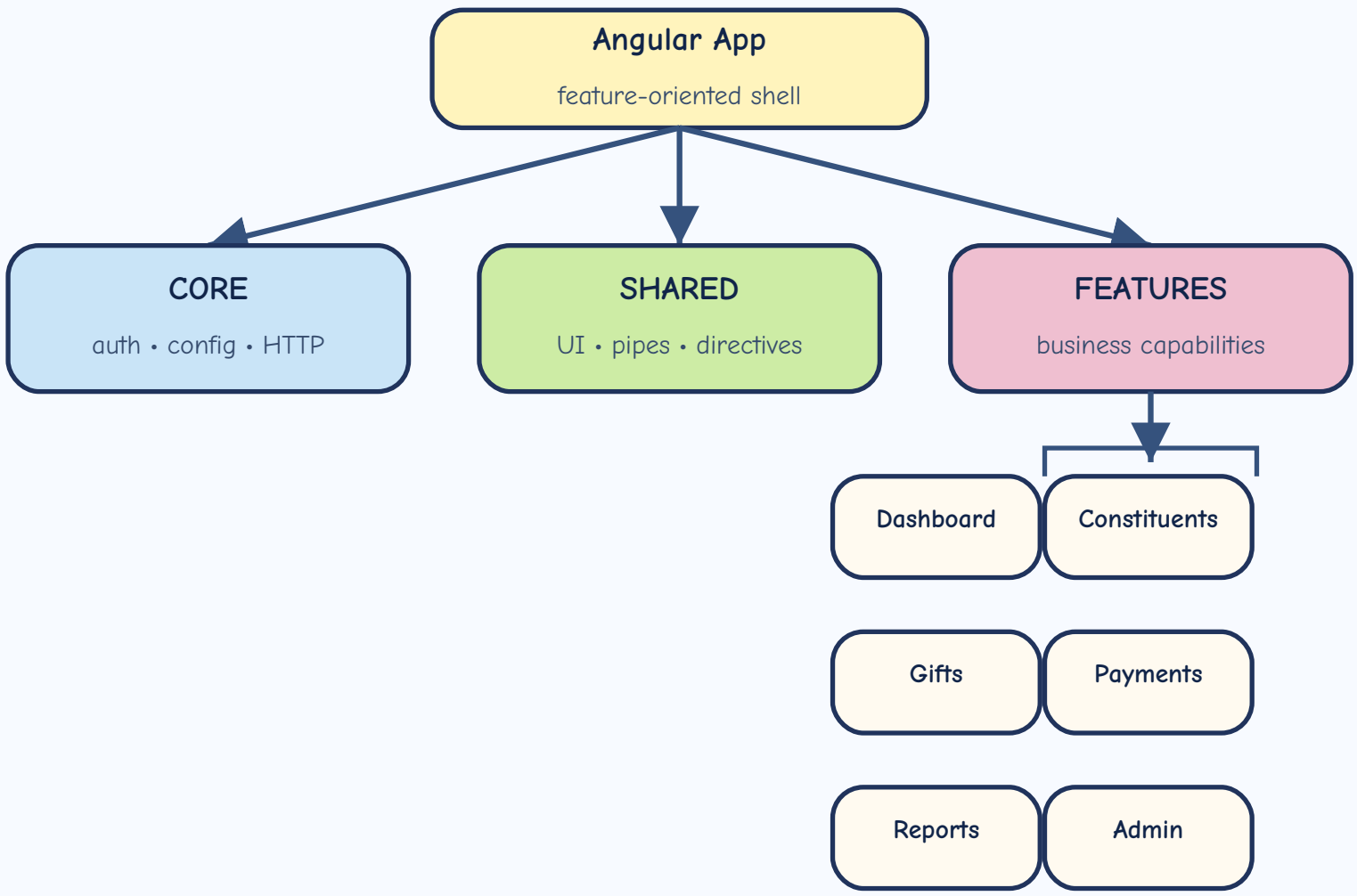
The application should remain in strict TypeScript mode.

Do not disable strictness merely because TypeScript produces errors.

Strict typing helps detect errors before production.

DIAGRAM

Feature-Oriented Angular Architecture



Core is app-wide infrastructure, Shared is reusable non-business UI, and Features own business-specific pages, models, state, and API clients.

4. High-Level Application Architecture

Use a feature-oriented architecture:

EXAMPLE / FLOW

```
src/
└── app/
    │
    ├── core/
    │
    ├── shared/
    │
    ├── layout/
    │
    ├── features/
    │   ├── dashboard/
    │   ├── constituents/
    │   ├── campaigns/
    │   ├── funds/
```

```
    │   ├── gifts/
    │   ├── interactions/
    │   ├── tasks/
    │   ├── payments/
    │   ├── settlements/
    │   ├── reports/
```

EXAMPLE / FLOW

```
    │   └── admin/
    │
    ├── app.config.ts
    ├── app.routes.ts
    └── app.ts
```

Do not organize the entire project like:

EXAMPLE / FLOW

```
components/
services/
models/
pages/
```

with hundreds of unrelated files in each folder.

Instead:

EXAMPLE / FLOW

```
feature
→ owns feature-specific components
→ owns feature API clients
→ owns feature models
→ owns feature state
→ owns feature routes
```

This makes the application easier to grow.

5. Core vs Shared vs Feature

core/

Contains application-wide infrastructure.

Example:

EXAMPLE / FLOW

```
core/
├── auth/
├── config/
├── http/
├── errors/
├── logging/
├── guards/
└── services/
```

Things in `core` normally exist once for the application.

Examples:

EXAMPLE / FLOW

```
AuthenticationService
CurrentUserService
TenantContext
GlobalErrorHandler
API configuration
```

shared/

Contains reusable UI pieces that are not business-specific.

Example:

EXAMPLE / FLOW

```
shared/
├── ui/
│   ├── button/
│   ├── dialog/
│   ├── table/
│   ├── pagination/
│   ├── loading/
│   ├── empty-state/
│   └── error-state/
│
├── pipes/
├── directives/
└── utilities/
```

Do not place:

EXAMPLE / FLOW

```
GiftFormComponent
```

inside `shared`.

It belongs to:

EXAMPLE / FLOW

`features/gifts`

because Gift is a business concept.

features/

Contains the actual business application.

Example:

EXAMPLE / FLOW

```
features/constituents/
├── pages/
├── components/
├── data-access/
├── models/
├── state/
├── constituent.routes.ts
└── constituent.models.ts
```

6. Complete Feature Structure

Recommended:

EXAMPLE / FLOW

```
features/
│
├── dashboard/
│
├── constituents/
│   ├── pages/
│   │   ├── constituent-list/
│   │   ├── constituent-details/
│   │   ├── constituent-create/
│   │   └── constituent-edit/
│   │
│   ├── components/
│   │   ├── constituent-summary/
│   │   ├── constituent-addresses/
│   │   ├── constituent-gifts/
```

```
│   │   ├── constituent-interactions/
│   │   └── constituent-tasks/
│   │
│   ├── data-access/
│   ├── state/
```

EXAMPLE / FLOW

```
│   └── constituent.routes.ts
│
├── campaigns/
├── funds/
├── gifts/
├── interactions/
├── tasks/
├── payments/
├── settlements/
├── reports/
└── admin/
```

7. Application Shell

The application shell contains:

EXAMPLE / FLOW

```
Top navigation
Side navigation
User menu
Tenant/business context
Breadcrumb
Main router outlet
Global notifications
```

Conceptually:

EXAMPLE / FLOW

```
+-----------------------------------------------------+
| Fundraising Platform              Sarah Johnson  ▼  |
+----------------+------------------------------------+
| Dashboard      |                                    |
| Constituents   |                                    |
| Campaigns      |           Router Outlet            |
| Funds          |                                    |
| Gifts          |                                    |
| Tasks          |                                    |
```

```
| Payments        |                                      |
| Reports         |                                      |
| Admin           |                                      |
+-----------------+--------------------------------------+
```

8. Routing

Routes represent major application navigation.

Example:

EXAMPLE / FLOW

```
/dashboard
/constituents
/constituents/:id
/campaigns
/campaigns/:id
/funds
/gifts
/gifts/:id
/tasks
/payments
/payments/:id
/settlements
/reports
/admin
```

9. Lazy Loading

Do not load the entire application JavaScript immediately.

Instead:

EXAMPLE / FLOW

```
Application Shell
      |
      +---- Dashboard
      |
      +---- Constituents
      |
      +---- Campaigns
      |
      +---- Payments
```

load feature code when needed.

Example:

TYPESCRIPT

```
export const routes: Routes = [
  {
    path: 'constituents',
    loadChildren: () =>
      import('./features/constituents/constituent.routes')
        .then(m => m.CONSTITUENT_ROUTES)
  },

  {
    path: 'payments',
    loadChildren: () =>
      import('./features/payments/payment.routes')
        .then(m => m.PAYMENT_ROUTES)
  }
];
```

Angular supports lazy-loaded standalone routes/components and provides tooling for converting eager routes to lazy ones.

Why Lazy Load?

Imagine:

EXAMPLE / FLOW

```
Fundraiser
```

never visits:

EXAMPLE / FLOW

```
Settlement Administration.
```

Why should that employee download all settlement UI code when opening the dashboard?

Lazy loading means:

EXAMPLE / FLOW

```
initial JavaScript smaller
+
faster startup
+
feature loaded when needed
```

10. Preloading

Lazy loading improves initial load, but the first visit to a feature can require another network request.

Angular supports preloading strategies such as:

EXAMPLE / FLOW

```
NoPreloading
PreloadAllModules
```

and custom strategies.

For this application, eventually create a selective strategy.

For example:

EXAMPLE / FLOW

```
preload:
Constituents
Campaigns
Tasks

do not immediately preload:
Admin
Settlement reports
rare advanced screens
```

11. @defer

Angular can defer expensive content inside a page.

Good dashboard example:

EXAMPLE / FLOW

```
Dashboard
   |
   +---- critical summary cards
   |
   +---- @defer large chart
   |
   +---- @defer secondary analytics
```

Example concept:

HTML

```
@defer (on viewport) {
  <app-campaign-performance-chart />
} @placeholder {
  <app-chart-skeleton />
}
```

Angular generates dynamic imports for eligible standalone dependencies inside `@defer`.

Do not defer:

EXAMPLE / FLOW

```
everything
```

because critical content should appear immediately.

12. Components

Angular components combine:

EXAMPLE / FLOW

```
TypeScript behavior
HTML template
CSS/SCSS styling
```

Example:

TYPESCRIPT

```typescript
@Component({
  selector: 'app-gift-summary',
  standalone: true,
  templateUrl: './gift-summary.html',
  styleUrl: './gift-summary.scss'
})
export class GiftSummaryComponent {
}
```

Smart/Page vs Presentational Components

A useful distinction:

EXAMPLE / FLOW

```
Page / container component
→ coordinates route/API/state

Presentational component
→ displays data and emits UI events
```

Example:

EXAMPLE / FLOW

```
GiftDetailsPage
      |
      +---- GiftSummaryCard
      |
      +---- GiftAllocationTable
      |
      +---- PaymentStatusCard
```

The child components do not all need to know about:

EXAMPLE / FLOW

```
HttpClient
Router
MSAL
```

13. Component Input

A child component may receive:

EXAMPLE / FLOW

```
GiftSummary
```

from its parent.

Modern Angular provides signal-based inputs.

Conceptually:

TYPESCRIPT

```
gift = input.required<GiftSummary>();
```

Then:

HTML

```
<h2>{{ gift().displayName }}</h2>
```

Use inputs to keep child components reusable.

14. Component Outputs

If a child needs to notify its parent:

EXAMPLE / FLOW

```
Edit button clicked
```

it should emit an event rather than directly manipulating parent internals.

Example concept:

TYPESCRIPT

```
editRequested = output<void>();
```

15. Signals

Signals are central to modern Angular.

A signal stores reactive state.

Example:

TYPESCRIPT

```
readonly searchText = signal('');
```

Read:

TYPESCRIPT

```
this.searchText();
```

Write:

TYPESCRIPT

```
this.searchText.set('John');
```

Update:

TYPESCRIPT

```
this.page.update(p => p + 1);
```

Where Signals Belong in This Project

Excellent uses:

EXAMPLE / FLOW

```
current UI filter
selected tab
dialog open/closed
loading state
current form model
selected rows
current user view state
feature-level local state
```

16. computed()

A computed signal derives state.

Example:

TYPESCRIPT

```
readonly allocationTotal = computed(() =>
  this.allocations()
    .reduce((sum, x) => sum + x.amount, 0)
);
```

Then:

TYPESCRIPT

```
readonly remainingAmount = computed(() =>
  this.giftAmount() - this.allocationTotal()
);
```

Business UI:

EXAMPLE / FLOW

```
Gift amount          $1,000
Allocated              $700
Remaining              $300
```

When allocation changes:

EXAMPLE / FLOW

```
remaining automatically recalculates.
```

This is an ideal Signals use.

17. effect()

`effect()` performs side effects when signal dependencies change.

Use sparingly.

Possible example:

EXAMPLE / FLOW

```
save certain non-sensitive UI preference
when it changes
```

Do **not** build a chain where:

EXAMPLE / FLOW

```
effect updates signal
→ effect updates another signal
→ effect updates another signal
```

for ordinary derived state.

Use:

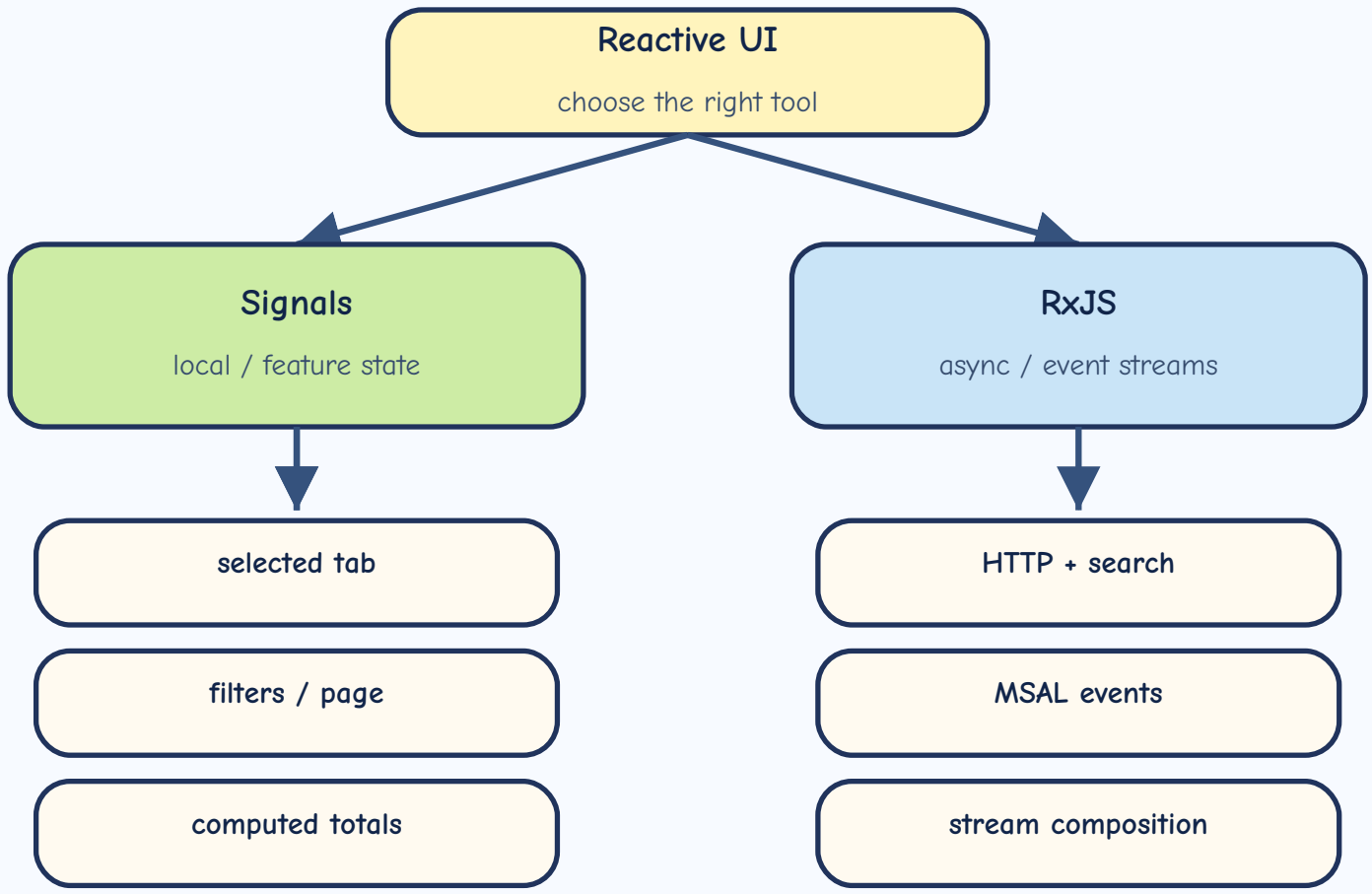
EXAMPLE / FLOW

```
computed()
```

for derived values.

DIAGRAM

Signals and RxJS Work Together



Signals are ideal for reactive UI state; RxJS remains the right tool for HTTP, debounced search, authentication events, and composed async streams.

18. Signals vs RxJS

Do not treat these as competitors.

Use:

EXAMPLE / FLOW

```
Signals
→ application/UI state
```

and:

EXAMPLE / FLOW

```
RxJS
→ streams/events/async composition
```

Examples:

Signals:

EXAMPLE / FLOW

```
selected campaign
current filter
current page
loading indicator
computed total
```

RxJS:

EXAMPLE / FLOW

```
HTTP response
debounced search input
MSAL events
route events
websocket/event streams
retry pipelines
```

19. RxJS Observable

An Observable represents values over time.

Example:

TYPESCRIPT

```
this.http.get<GiftResponse>(url)
```

returns an observable.

It may:

EXAMPLE / FLOW

```
emit result
error
complete
```

Important Operators

You should learn:

EXAMPLE / FLOW

```
map
filter
```

```
tap
switchMap
mergeMap
concatMap
exhaustMap
debounceTime
distinctUntilChanged
catchError
finalize
combineLatest
forkJoin
shareReplay
takeUntilDestroyed
```

Each should be learned through this project.

20. switchMap

Perfect for search.

User types:

EXAMPLE / FLOW

```
J
Jo
Joh
John
```

You don't want four old searches racing.

Use:

EXAMPLE / FLOW

```
debounceTime
+
distinctUntilChanged
+
switchMap
```

Conceptually:

TYPESCRIPT

```
search$
  .pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(search =>
      constituentApi.search({ search })
    )
  );
```

`switchMap` cancels the previous subscription when a new search arrives.

Good for:

EXAMPLE / FLOW

```
search
filtering
route-driven data
```

21. concatMap

Useful when operations must happen sequentially.

Example:

EXAMPLE / FLOW

```
upload/import steps
```

where ordering matters.

22. mergeMap

Useful when operations can run concurrently.

But uncontrolled concurrency can overload your API.

Do not automatically use:

EXAMPLE / FLOW

```
mergeMap
```

for hundreds of write operations.

23. exhaustMap

Excellent for preventing repeated submissions.

Imagine user double-clicks:

EXAMPLE / FLOW

```
Create Payment
```

`exhaustMap` can ignore further submit triggers while the first operation is still running.

Still keep server-side idempotency.

Frontend protection is not enough for financial operations.

24. takeUntilDestroyed

When manually subscribing in components/services, Angular provides:

EXAMPLE / FLOW

```
takeUntilDestroyed()
```

which completes the subscription when the owning context is destroyed, avoiding the old pattern of manually maintaining a destruction `Subject`.

Example:

TYPESCRIPT

```ts
this.authEvents$
  .pipe(takeUntilDestroyed())
  .subscribe(...);
```

25. Avoid Manual Subscribe When Possible

Instead of:

TYPESCRIPT

```ts
this.api.getGifts().subscribe(x => {
  this.gifts = x;
});
```

you can often use:

EXAMPLE / FLOW

```
signals/resources
AsyncPipe
toSignal
structured feature state
```

depending on the situation.

Manual subscriptions are valid but should have a reason.

26. State Management Strategy

Do not install a huge state framework on day one simply because the application is enterprise-sized.

Use three levels.

EXAMPLE / FLOW

```
Level 1
Component state
→ signals

Level 2
Feature state
→ injectable signal-based store/service

Level 3
Cross-application complex state
→ state library only if genuinely needed
```

Component State Example

Gift form:

EXAMPLE / FLOW

```
selected campaign
allocation rows
dialog open state
```

keep locally.

Feature State Example

Constituent feature:

EXAMPLE / FLOW

```
search filters
current result page
selected constituent
loading state
```

can live in:

EXAMPLE / FLOW

```
ConstituentStore
```

27. Simple Signal Store

Concept:

TYPESCRIPT

```
@Injectable()
export class ConstituentStore {
  private readonly _items =
    signal<readonly ConstituentSummary[]>([]);

  private readonly _loading =
    signal(false);

  readonly items =
    this._items.asReadonly();

  readonly loading =
    this._loading.asReadonly();
}
```

Methods:

EXAMPLE / FLOW

```
load()
search()
clear()
```

The store owns feature state.

Do We Need NgRx?

Learn NgRx because it is common in enterprise Angular.

Do not make it mandatory for this first implementation.

Use NgRx when you actually need:

EXAMPLE / FLOW

```
large shared state graph
complex event-driven state
developer tooling
predictable reducers/effects
cross-feature coordination
```

For most CRUD screens:

EXAMPLE / FLOW

```
signals
+
feature services
+
HttpClient
```

are sufficient.

A later refactoring exercise can implement:

EXAMPLE / FLOW

```
Payment Operations
```

or:

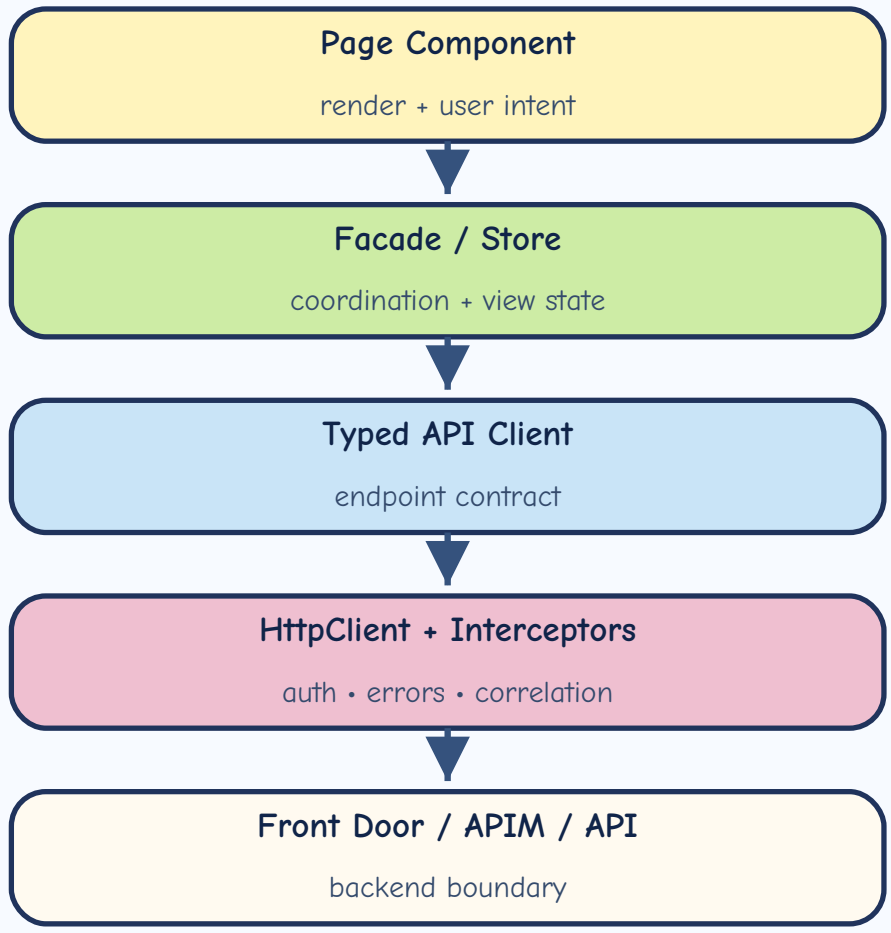
EXAMPLE / FLOW

```
Dashboard
```

using NgRx/Signal Store for comparison.

DIAGRAM

Frontend HTTP Responsibility Layers



Components should not coordinate raw HTTP directly. A facade/store owns feature behavior, typed API clients own contracts, and interceptors handle cross-cutting concerns.

28. HTTP Client Architecture

Do not inject raw `HttpClient` into every component.

Instead:

EXAMPLE / FLOW

```
Component
    |
    v
Feature Store / Facade
    |
    v
API Client
    |
    v
HttpClient
```

Example:

EXAMPLE / FLOW

```
GiftDetailsPage
      |
      v
GiftStore
      |
      v
GiftApiClient
      |
      v
HttpClient
```

29. Fundraising API Client

Example concept:

TYPESCRIPT

```ts
@Injectable({ providedIn: 'root' })
export class GiftApiClient {
  private readonly http = inject(HttpClient);
  private readonly config = inject(ApiConfiguration);

  getById(id: number) {
    return this.http.get<GiftResponse>(
      `${this.config.fundraisingBaseUrl}/gifts/${id}`
    );
  }
}
```

Separate API Clients

Use:

EXAMPLE / FLOW

```
ConstituentApiClient
CampaignApiClient
FundApiClient
GiftApiClient
TaskApiClient
PaymentApiClient
SettlementApiClient
ReportingApiClient
```

Do not create:

EXAMPLE / FLOW

```
ApiService
```

with 140 unrelated methods.

30. HttpClient Configuration

Modern Angular uses:

TYPESCRIPT

```
provideHttpClient(
  withInterceptors([
    ...
  ])
)
```

Functional interceptors are currently recommended because their behavior and ordering are more predictable in complex configurations.

31. Interceptors

Interceptors are frontend middleware around HTTP.

Use them for cross-cutting concerns.

Possible interceptors:

EXAMPLE / FLOW

```
Authentication/token integration
Correlation ID
Global API error normalization
Loading activity
Request timing
Retry only where appropriate
```

Angular's HttpClient interceptors are explicitly intended for cross-cutting concerns such as authentication, logging, caching, retrying, timeouts, and loading indicators.

Do Not Retry Everything

Never blindly retry:

EXAMPLE / FLOW

```
POST Payment
POST Refund
POST Gift
```

because writes may have already reached the server.

The server-side:

EXAMPLE / FLOW

```
idempotency design
```

is what ultimately makes retries safe where required.

32. Environment/API Configuration

Do not hard-code:

TYPESCRIPT

```
'https://some-dev-url.azure...'
```

inside services.

Create a configuration abstraction:

TYPESCRIPT

```
export interface AppConfiguration {
  fundraisingApiUrl: string;
  paymentApiUrl: string;
  reportingApiUrl: string;
}
```

In production, with APIM, these may share one public base:

EXAMPLE / FLOW

```
https://api.example.com
```

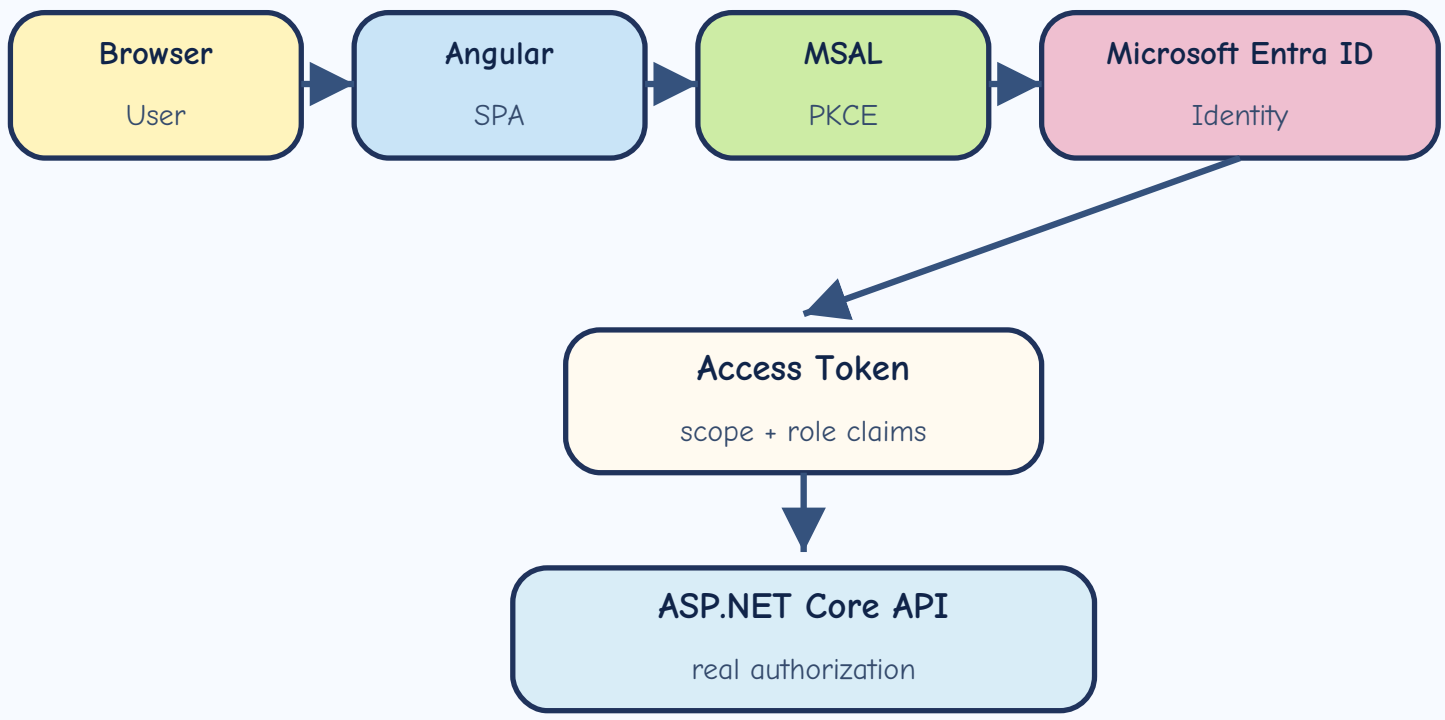
and use routes:

EXAMPLE / FLOW

```
/fundraising/...
/payments/...
/reports/...
```

DIAGRAM

Authorization Code + PKCE Authentication Flow



Route guards improve UX; backend policies remain the security authority.

MSAL obtains an access token from Microsoft Entra ID; Angular sends it to the API, which validates identity and enforces authorization policies.

33. Microsoft Entra + MSAL

Authentication architecture:

EXAMPLE / FLOW

```
Angular
   |
   v
MSAL
   |
   v
Microsoft Entra ID
   |
   v
Authorization Code + PKCE
   |
   v
Access Token
   |
   v
Web API
```

Single-page applications should use OAuth 2.0 Authorization Code flow with PKCE; the old implicit grant is no longer recommended. MSAL Browser implements Authorization Code + PKCE for browser SPAs.

34. MSAL Packages

Conceptually install:

EXAMPLE / FLOW

```
@azure/msal-browser
@azure/msal-angular
```

Use versions compatible with the Angular version in your project.

35. Entra Configuration

Frontend configuration needs public identifiers such as:

EXAMPLE / FLOW

```
Client ID
Tenant ID / authority
Redirect URI
API scopes
```

These values are not secrets.

Example:

TYPESCRIPT

```ts
export const authConfig = {
  clientId: '...',
  authority:
    'https://login.microsoftonline.com/<tenant-id>',
  redirectUri:
    window.location.origin
};
```

Never put:

EXAMPLE / FLOW

```
client secret
```

in Angular.

A browser cannot keep a secret.

36. Login Flow

EXAMPLE / FLOW

```
User opens app
      |
      v
MSAL checks account
      |
      +---- signed in
      |        |
      |        v
      |   app continues
      |
      +---- not signed in
               |
               v
        redirect/popup login
               |
               v
        Microsoft Entra ID
               |
               v
        callback to Angular
```

37. Access Token Flow

When Angular calls:

EXAMPLE / FLOW

```
GET /api/v1/constituents
```

MSAL obtains an access token for:

EXAMPLE / FLOW

```
Fundraising API scope.
```

HTTP request:

EXAMPLE / FLOW

```
Authorization:
Bearer ey...
```

Then API validates it.

38. Token Acquisition

MSAL handles token caching and silent acquisition where possible.

Do not write:

EXAMPLE / FLOW

```
localStorage.setItem(
  'my-token',
  token
);
```

manually throughout the application.

Use MSAL APIs.

39. MSAL Guard

Protect application routes such as:

EXAMPLE / FLOW

```
/dashboard
/constituents
/gifts
```

using authentication guards.

MSAL Angular provides guard capabilities for protected Angular routes and interceptor integration for acquiring tokens for API requests.

40. Authorization Guard

Authentication only means:

EXAMPLE / FLOW

```
user is signed in.
```

You also need role authorization.

Example:

EXAMPLE / FLOW

```
/payments/refunds
```

requires:

EXAMPLE / FLOW

```
FinanceUser
or
PaymentOperations
```

Create route guards based on:

EXAMPLE / FLOW

```
claims
roles
```

Important Security Rule

Route guard:

EXAMPLE / FLOW

```
improves UX.
```

It does **not** secure the API.

A user can bypass browser UI and call HTTP directly.

Real authorization remains:

EXAMPLE / FLOW

```
ASP.NET Core authorization policy.
```

41. Role-Based Navigation

Sarah is a:

EXAMPLE / FLOW

```
Fundraiser
```

Her navigation might show:

EXAMPLE / FLOW

```
Dashboard
Constituents
Campaigns
Funds
Gifts
Tasks
Reports
```

but not:

EXAMPLE / FLOW

```
Refund Administration
```

Finance User might additionally see:

EXAMPLE / FLOW

```
Payments
Settlements
```

Use roles/permissions to construct the menu.

Do Not Hard-Code Security Only in Templates

Bad:

HTML

```
@if (user.email === 'admin@company.com') {
  <button>Refund</button>
}
```

Better:

EXAMPLE / FLOW

```
permission service
→ canRefundPayment()
```

backed by authenticated claims/roles.

And the backend still verifies permission.

42. Current User State

Create an authentication/current-user facade.

Concept:

TYPESCRIPT

```
@Injectable({ providedIn: 'root' })
export class CurrentUserService {
  readonly account = signal<AccountInfo | null>(null);

  readonly isAuthenticated =
    computed(() => this.account() !== null);

  readonly roles =
    computed(() => ...);
}
```

This is UI convenience state.

Do not use this as the backend security authority.

43. Tenant Context

After login:

EXAMPLE / FLOW

```
Entra identity
      |
      v
GET /api/v1/users/me
      |
      v
AppUser
      |
      v
Application TenantId
```

The frontend may display:

EXAMPLE / FLOW

```
ABC Children's Foundation
```

but should not be allowed to arbitrarily send:

EXAMPLE / FLOW

```
TenantId = 999
```

and switch security context.

The backend derives/validates tenant context.

44. Forms Strategy

Angular 22 gives us three options:

EXAMPLE / FLOW

```
Signal Forms
Reactive Forms
Template-driven Forms
```

Angular's current guidance lists Signal Forms as stable in v22+, while Reactive Forms remain a stable choice, especially for existing applications.

For this new project:

EXAMPLE / FLOW

```
Primary approach
→ Signal Forms
```

Also learn:

EXAMPLE / FLOW

```
Reactive Forms
```

because many enterprise Angular applications use them.

Use template-driven forms only for simple educational cases.

45. Signal Forms

Signal Forms use a writable signal as the form model.

Example:

TYPESCRIPT

```ts
interface GiftFormModel {
  constituentId: number | null;
  campaignId: number | null;
  giftDate: string;
  totalAmount: number;
  currencyCode: string;
}

readonly giftModel = signal<GiftFormModel>({
  constituentId: null,
  campaignId: null,
  giftDate: '',
  totalAmount: 0,
```

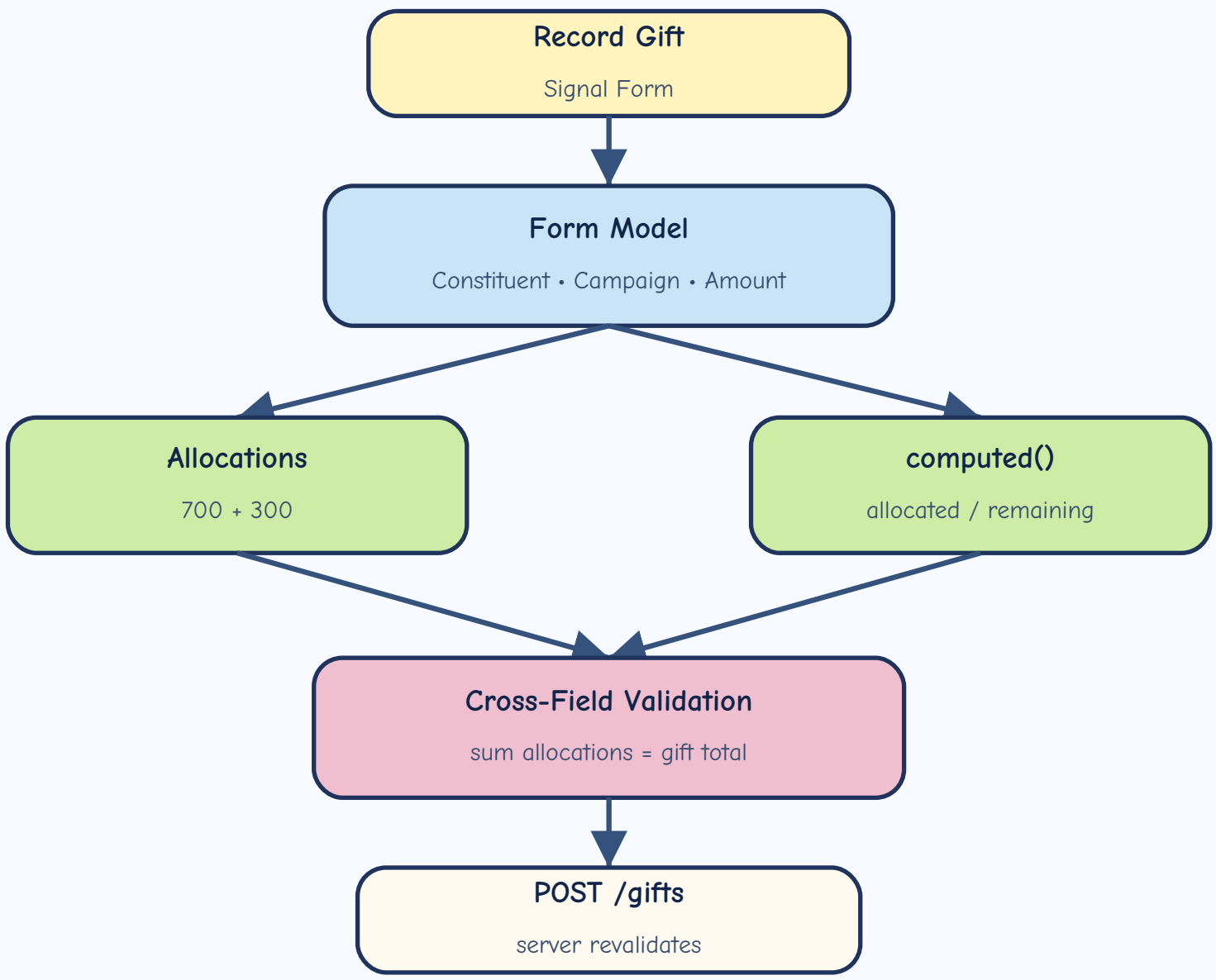
```
  currencyCode: 'USD'
});
```

Then create the form around that model.

Signal Forms provide type-safe field trees and schema-based validation.

DIAGRAM

Gift Form - Reactive Validation Flow



The frontend provides immediate allocation feedback, while .NET and SQL remain authoritative for business integrity.

46. Gift Form

The Gift form should contain:

EXAMPLE / FLOW

```
Constituent
Campaign
Gift Date
Amount
Currency
Gift Type
Reference Number
Notes
Allocations
```

UI:

EXAMPLE / FLOW

```
------------------------------------------------
Record Gift
------------------------------------------------

Constituent
[ John Smith                                ]

Campaign
[ 2026 Cancer Research Campaign             ]

Gift Date
[ 09/23/2026 ]

Amount
[ 1000.00 ]

Currency
[ USD ]

Allocations
```

EXAMPLE / FLOW

```
------------------------------------------------
Cancer Research                  $700
Patient Assistance               $300
------------------------------------------------
Allocated                      $1,000
Remaining                          $0

                    [ Cancel ] [ Save Gift ]
```

47. Cross-Field Validation

This is extremely important.

Business rule:

EXAMPLE / FLOW

```
SUM(allocations.amount)
=
Gift.TotalAmount
```

Frontend should detect:

EXAMPLE / FLOW

```
Gift total = $1,000
Allocated = $900
```

and display:

EXAMPLE / FLOW

```
$100 remains to be allocated.
```

Signal Forms supports cross-field logic and validators that depend on other field/model values.

Still validate the same rule in:

EXAMPLE / FLOW

```
.NET
and
SQL transaction
```

Frontend validation is UX, not final integrity.

48. Reactive Forms

You should also implement one substantial form with traditional typed Reactive Forms.

Good choice:

EXAMPLE / FLOW

```
Campaign Edit
```

Learn:

EXAMPLE / FLOW

```
FormControl
FormGroup
FormArray
Validators
valueChanges
statusChanges
cross-field validators
typed forms
```

Reactive Forms remain strictly typed by default.

49. FormArray

Useful for:

EXAMPLE / FLOW

```
Gift Allocations
```

in Reactive Forms.

Concept:

EXAMPLE / FLOW

```
Allocations
 |
 +---- Fund 300 / 700
 |
 +---- Fund 301 / 300
```

Each allocation is a form row.

50. Validation Levels

Frontend validation:

EXAMPLE / FLOW

```
required
format
length
simple cross-field validation
```

Backend:

EXAMPLE / FLOW

```
authorization
business rules
current database state
concurrency
```

SQL Server:

EXAMPLE / FLOW

```
constraints
transactional integrity
```

Use all three.

51. Server Validation Errors

Suppose API returns:

JSON

```
{
  "title": "Validation failed.",
  "status": 400,
  "errors": {
    "primaryEmail": [
      "Email already exists."
    ]
  }
}
```

Angular should map that error back to:

EXAMPLE / FLOW

```
Primary Email field
```

when possible.

Do not show every failure as:

EXAMPLE / FLOW

```
Something went wrong.
```

52. ProblemDetails Client Model

Your .NET API uses:

EXAMPLE / FLOW

```
ProblemDetails
```

Create TypeScript representation:

TYPESCRIPT

```
export interface ApiProblem {
  type?: string;
  title?: string;
  status?: number;
  detail?: string;
  traceId?: string;
  errors?: Record<string, string[]>;
}
```

This gives consistent error handling.

53. Global HTTP Error Handling

Use an HTTP interceptor/facade for common responses:

EXAMPLE / FLOW

```
401
→ authentication handling

403
→ access denied

429
→ rate limited

500
→ generic failure notification
```

But business-specific errors should remain with the feature.

Example:

EXAMPLE / FLOW

```
Gift.InvalidAllocationTotal
```

belongs in the Gift screen, not only a global toast.

54. Loading States

Every network screen needs:

EXAMPLE / FLOW

```
loading
success
empty
error
```

states.

Example:

EXAMPLE / FLOW

```
Constituent list

Loading
→ skeleton

Loaded with data
→ table

Loaded empty
→ "No constituents found"

Failed
→ error state + retry
```

Do not leave users looking at a blank screen.

55. Skeleton Loading

Use skeletons for major pages rather than only:

EXAMPLE / FLOW

```
spinning icon forever.
```

Example:

EXAMPLE / FLOW

```
Constituent Profile
----------------------
[redacted]
```

This gives the interface a stable layout during loading.

56. Notifications

Use:

EXAMPLE / FLOW

```
toast/snackbar
```

for transient actions:

EXAMPLE / FLOW

```
Gift created successfully
Task completed
Campaign updated
```

Do not put critical errors only in a disappearing toast.

57. Confirmation Dialogs

Require confirmation for destructive or financially significant actions.

Examples:

EXAMPLE / FLOW

```
Archive Constituent
Cancel Gift
Request Refund
Close Campaign
```

Refund confirmation should display:

EXAMPLE / FLOW

```
Payment
Original amount
Already refunded
Refund request amount
Remaining amount
```

before submission.

58. Concurrency in Angular

The API uses:

EXAMPLE / FLOW

```
rowversion
```

for optimistic concurrency.

Angular receives:

JSON

```
{
  "giftId": 10000,
  "amount": 1000,
  "rowVersion": "AAAAAAAAB9E="
}
```

On update Angular returns the original version.

If another user already changed the record:

EXAMPLE / FLOW

```
API
→ 409 Conflict
```

UI should say:

EXAMPLE / FLOW

```
This record was modified by another user.

Reload the latest version before saving again.
```

Buttons:

EXAMPLE / FLOW

```
[Reload Latest]

[Cancel]
```

Do not silently overwrite the other user's changes.

59. Lists and Tables

Main application tables:

EXAMPLE / FLOW

```
Constituents
Campaigns
Funds
Gifts
Tasks
Payments
Settlements
Reports
```

Every large table should support:

EXAMPLE / FLOW

```
server-side pagination
server-side filtering
server-side sorting
```

not load everything into the browser.

60. Pagination

Bad:

EXAMPLE / FLOW

```
GET 1,000,000 Gifts

Angular slices 50 at a time
```

Correct:

EXAMPLE / FLOW

```
Angular asks for 50

API asks SQL for 50
```

Flow:

EXAMPLE / FLOW

```
Angular
?pageSize=50
        |
        v
.NET API
        |
        v
SQL pagination
        |
        v
50 rows
```

61. Pagination Model

TYPESCRIPT

```ts
export interface PagedResult<T> {
  items: readonly T[];
  nextToken?: string;
}
```

or page-number version:

TYPESCRIPT

```ts
export interface PageResult<T> {
  items: readonly T[];
  page: number;
  pageSize: number;
  totalCount: number;
}
```

Which Pagination?

Learn both.

EXAMPLE / FLOW

```
OFFSET/page number
→ easy admin tables

Keyset/cursor
→ large timelines/high-volume data
```

Gift history and payment transaction feeds are strong candidates for cursor-based pagination.

62. Filtering

Constituent list:

EXAMPLE / FLOW

```
Search Name
Status
Type
```

Gift list:

EXAMPLE / FLOW

```
Constituent
Campaign
From Date
To Date
Minimum Amount
```

Payment list:

EXAMPLE / FLOW

```
Payment Status
Date
Provider
Amount
Gift ID
```

63. URL Query Parameters

Filters should often be represented in the URL.

Example:

EXAMPLE / FLOW

```
/constituents?search=john&status=active&page=2
```

Benefits:

EXAMPLE / FLOW

```
bookmarkable
browser back/forward
shareable
refresh-safe
```

Do not keep every search/filter only in hidden component state.

64. Sorting

Example:

EXAMPLE / FLOW

```
GET /gifts
?sort=giftDate
&direction=desc
```

The backend must whitelist supported sorting columns.

Angular should not send arbitrary SQL property names.

65. Search Debouncing

For text search:

EXAMPLE / FLOW

```
user types
```

do not send API request on every keystroke.

Use:

EXAMPLE / FLOW

```
300–500 ms debounce
+
distinctUntilChanged
+
switchMap
```

66. Dashboard

The main dashboard can include:

EXAMPLE / FLOW

```
Giving This Month
Giving This Year
Active Campaigns
Open Tasks
Overdue Tasks
Payment Success Rate
Failed Payments
Pending Refunds
```

Below:

EXAMPLE / FLOW

```
Monthly Giving Trend
Campaign Performance
Fund Distribution
Recent Gifts
My Tasks
```

Most analytical data should come from:

EXAMPLE / FLOW

```
Reporting API
```

rather than performing hundreds of Fundraising API calls from Angular.

67. Dashboard API Design

Bad Angular:

EXAMPLE / FLOW

```
GET gifts
GET campaigns
GET funds
GET payments
GET refunds
load everything
calculate dashboard in browser
```

Better:

EXAMPLE / FLOW

```
GET /reports/executive
```

returns purpose-built dashboard summary.

The server/database should aggregate where appropriate.

68. Constituents Feature

Main screens:

EXAMPLE / FLOW

```
Constituent List
Create Constituent
Constituent Details
Edit Constituent
```

Detail workspace:

EXAMPLE / FLOW

```
John Smith

[Overview]
[Addresses]
[Gifts]
[Interactions]
[Tasks]
[Timeline]
```

Constituent Overview

Display:

EXAMPLE / FLOW

```
Name
Email
Phone
Primary address
Status
Lifetime giving
Last gift
Open tasks
Last interaction
```

Constituent Timeline

Timeline combines:

EXAMPLE / FLOW

```
Gift
Interaction
Task
```

API already gives a combined timeline.

Angular only renders:

EXAMPLE / FLOW

```
Jan 10  Phone Call
Jan 15  Gift $1,000
Jan 16  Thank-you Task
```

Do not call three APIs and reproduce SQL `UNION ALL` business logic in Angular unless the contract explicitly requires client aggregation.

69. Campaign Feature

Screens:

EXAMPLE / FLOW

```
Campaign List
Campaign Create
Campaign Detail
Campaign Edit
Campaign Performance
Campaign History
```

Detail:

EXAMPLE / FLOW

```
Cancer Research Campaign

Goal            $500,000
Raised          $375,000
Remaining       $125,000
Progress        75%
```

Progress Component

Reusable component:

EXAMPLE / FLOW

```
<app-progress
  [value]="raised"
  [maximum]="goal"
/>
```

Can be shared by:

EXAMPLE / FLOW

```
Campaign
Fundraising dashboard
```

but keep the component generic.

Campaign History

Temporal history endpoint:

EXAMPLE / FLOW

```
GET /campaigns/{id}/history
```

UI:

EXAMPLE / FLOW

```
Date                Goal
--------------------------------
Jan 01              $400,000
Mar 15              $450,000
Jun 01              $500,000
```

This connects your SQL Temporal Table work to Angular.

70. Funds Feature

Screens:

EXAMPLE / FLOW

```
Fund List
Fund Detail
Create Fund
Edit Fund
Fund Giving
```

Display:

EXAMPLE / FLOW

```
Restricted
Unrestricted
```

clearly.

71. Gift Feature

Gift area is one of the most important features.

Screens:

EXAMPLE / FLOW

```
Gift List
Record Gift
Gift Detail
Edit Gift
Gift Allocations
Payment Status
```

Gift Detail Layout

EXAMPLE / FLOW

```
Gift #10000

Donor
John Smith

Amount
$1,000

Campaign
Cancer Research 2026

---------------------------------

Allocations

Cancer Research          $700

Patient Assistance       $300

---------------------------------
```

EXAMPLE / FLOW

```
Payment
Succeeded

---------------------------------

Recorded By
Sarah Johnson
```

72. Tasks Feature

Screens:

EXAMPLE / FLOW

```
My Tasks
All Tasks
Overdue Tasks
Task Detail
Create Task
```

Filters:

EXAMPLE / FLOW

```
Assigned To
Priority
Due Date
Completed/Open
```

Kanban?

You can build an optional:

EXAMPLE / FLOW

```
Task board
```

later:

EXAMPLE / FLOW

```
Open
In Progress
Completed
```

but only after the underlying Task lifecycle supports those states.

Do not invent frontend states that your API/domain does not contain.

73. Payments Feature

Access limited to appropriate roles.

Screens:

EXAMPLE / FLOW

```
Payment Search
Payment Detail
Payment Attempts
Refunds
Webhook/operations view
Settlements
```

Payment Detail

EXAMPLE / FLOW

```
Payment #90001

Gift
10000

Amount
$1,000

Status
Succeeded

Method
Visa ****4242

Provider
Provider A

Attempts
3
```

EXAMPLE / FLOW

```
Refunded
$200

Net
$800
```

74. Payment Attempts

Timeline:

EXAMPLE / FLOW

```
Attempt 1
Declined
9:03 AM

Attempt 2
Timeout
9:05 AM

Attempt 3
Succeeded
9:07 AM
```

This helps operations understand payment history.

75. Refund Screen

Only permitted roles see:

EXAMPLE / FLOW

```
Refund
```

Form:

EXAMPLE / FLOW

```
Original Payment        $1,000
Already Refunded          $200
Available                 $800

Refund Amount
[ 100.00 ]

Reason
[ Duplicate donation ]

[Cancel] [Submit Refund]
```

Important Refund Rules

Frontend can check:

EXAMPLE / FLOW

```
refund <= visible available amount
```

but backend must recheck because:

EXAMPLE / FLOW

```
another user may refund simultaneously.
```

API concurrency is authoritative.

76. Settlement Feature

Finance views:

EXAMPLE / FLOW

```
Settlement list
Settlement detail
Included payments
Gross amount
Fees
Net amount
Reconciliation status
```

Example:

EXAMPLE / FLOW

```
Settlement #4501

Gross          $100,000
Fees             $3,000
Net             $97,000

Payments            842
```

77. Reporting Feature

Reports:

EXAMPLE / FLOW

```
Executive Dashboard
Monthly Giving
Campaign Performance
Fund Performance
Top Donors
Payment Performance
Refund Analysis
Interaction Activity
```

Use Reporting API.

Charts

Chart examples:

EXAMPLE / FLOW

```
Monthly giving
→ line/bar chart

Campaign totals
→ bar chart

Fund distribution
→ bar/donut if appropriate

Payment success/failure
→ bar/percentage visualization
```

Always include accessible numerical/table alternatives for important analytics.

Charts should support understanding; they should not be the only way to access data.

78. Admin Feature

Admin screens can include:

EXAMPLE / FLOW

```
Users
Roles
Tenant settings
Integration health
Failed outbox messages
Failed webhook events
System status
```

Do **not** expose:

EXAMPLE / FLOW

```
database passwords
Key Vault secrets
raw access tokens
```

through the admin UI.

79. Design System

Do not create a different:

EXAMPLE / FLOW

```
button
table
dialog
form style
```

in every feature.

Build reusable design primitives.

Examples:

EXAMPLE / FLOW

`AppButton`
`AppDialog`
`AppTable`
`AppBadge`
`AppCard`
`AppEmptyState`
`AppErrorState`
`AppSkeleton`
`AppPagination`

Alternatively use a supported UI component library/design system and add project-specific wrappers where useful.

80. Angular Material

Angular Material is a reasonable option for this enterprise-style application.

Useful components:

EXAMPLE / FLOW

`buttons`
`dialogs`
`menus`
`form fields`
`tables`
`paginator`
`sidenav`
`tabs`
`snackbar`
`tooltips`

Do not let a component library decide your business architecture.

81. Accessibility

Accessibility is not optional.

Cover:

EXAMPLE / FLOW

```
semantic HTML
labels
keyboard navigation
focus management
ARIA where required
color contrast
screen-reader feedback
accessible validation
```

Angular 22 also has stable Angular Aria primitives for accessible headless interaction patterns.

Forms Accessibility

Every field needs:

EXAMPLE / FLOW

```
label
error association
required indication
keyboard accessibility
```

Bad:

HTML

```
<input placeholder="Email">
```

with no accessible label.

Better:

HTML

```
<label for="email">Email</label>
<input id="email" ...>
```

Dialog Accessibility

When opening a dialog:

EXAMPLE / FLOW

```
focus enters dialog
keyboard remains usable
Escape behavior appropriate
focus returns to original element
```

Use a mature accessible dialog implementation rather than inventing one casually.

82. Change Detection

Angular 22 defaults to `OnPush`, and Angular v21+ defaults to zoneless change detection.

This means your application should naturally favor:

EXAMPLE / FLOW

```
signals
AsyncPipe
immutable-style state updates
Angular APIs that notify state changes
```

rather than manually forcing broad change detection.

Zoneless

Old Angular relied heavily on:

EXAMPLE / FLOW

```
Zone.js
```

to notice asynchronous work.

Modern Angular can update efficiently through explicit reactive notifications such as signal changes and template events. The Angular team cites lower change-detection overhead, smaller payloads, improved debugging, and better compatibility among the reasons for zoneless operation.

For this new project:

EXAMPLE / FLOW

```
stay with the Angular 22 zoneless default.
```

83. Performance Rules

Lazy-load features

Already covered.

Use @defer for non-critical expensive UI

Example:

EXAMPLE / FLOW

```
large analytics charts
```

Server-side pagination

Never download massive tables.

Project API Responses

Avoid requesting fields not needed by the page.

Example list:

EXAMPLE / FLOW

```
ConstituentId
DisplayName
Email
Status
```

Detail page can request more.

track in @for

When rendering lists:

HTML

```
@for (gift of gifts(); track gift.id) {
  ...
}
```

Stable identity helps Angular reuse DOM appropriately.

Avoid Expensive Template Functions

Bad:

HTML

```
{{ calculateHugeReport(gifts()) }}
```

on every rendering pass.

Prefer:

EXAMPLE / FLOW

```
computed signal
```

or preprocessed data.

Avoid Browser-Side Large Sorting

For 100,000 payment rows:

EXAMPLE / FLOW

```
sorting belongs on server.
```

Avoid Unnecessary Subscriptions

Use signals and RxJS interop correctly.

Avoid Memory Leaks

Use:

EXAMPLE / FLOW

```
takeUntilDestroyed()
```

when manual subscriptions are necessary.

84. Virtual Scrolling

If a UI legitimately needs a long rendered list, learn:

EXAMPLE / FLOW

```
virtual scrolling.
```

Only visible rows are rendered.

However:

EXAMPLE / FLOW

```
virtual scrolling
≠ replacement for server pagination.
```

Use both where appropriate.

85. State Persistence

Safe candidates:

EXAMPLE / FLOW

```
preferred table page size
UI theme
collapsed sidebar
```

Do not store:

EXAMPLE / FLOW

```
payment secret
access credentials
sensitive donor records
```

in localStorage simply for convenience.

MSAL manages authentication cache behavior through its supported APIs/configuration.

86. Browser Storage

Understand:

EXAMPLE / FLOW

```
localStorage
sessionStorage
IndexedDB
```

but use them deliberately.

Browser storage is readable by browser-side JavaScript, so XSS security matters.

Do not assume:

EXAMPLE / FLOW

`localStorage = secure secret vault.`

87. Security — XSS

Do not dynamically inject untrusted HTML.

Avoid:

EXAMPLE / FLOW

`innerHTML`

unless the content and sanitization model are explicitly understood.

Angular provides built-in template escaping/sanitization protections, but developers can bypass them with dangerous APIs.

Do not casually use:

EXAMPLE / FLOW

`bypassSecurityTrustHtml`

for database/user content.

88. CSRF/XSRF

With Bearer-token APIs where the Authorization header is explicitly attached, the threat model differs from cookie-authenticated applications.

Still understand Angular's built-in XSRF mechanisms because many enterprise applications use cookie-based backends.

Your chosen Entra/MSAL architecture should follow the actual authentication design rather than disabling protections without analysis.

89. CORS

Angular development:

EXAMPLE / FLOW

`http://localhost:4200`

calling:

EXAMPLE / FLOW

```
https://localhost:7001
```

may require CORS on ASP.NET Core.

Production behind Front Door/APIM should use explicitly allowed origins.

Do not solve CORS by saying:

EXAMPLE / FLOW

```
AllowAnyOrigin forever.
```

90. Correlation ID

When a request fails:

EXAMPLE / FLOW

```
Angular
      |
      v
APIM
      |
      v
Fundraising API
      |
      v
SQL
```

you need to correlate logs.

If the backend returns:

EXAMPLE / FLOW

```
traceId
```

inside ProblemDetails, display/capture it appropriately.

Example:

EXAMPLE / FLOW

```
Something went wrong.

Reference:
00-a63f...
```

This helps support teams locate the matching server trace.

91. API Versioning

Your backend routes use:

EXAMPLE / FLOW

```
/api/v1/...
```

Angular API clients should centralize versioning.

Example:

TYPESCRIPT

```
const API_VERSION = 'v1';
```

or configuration.

Do not scatter:

EXAMPLE / FLOW

```
'/v1/'
```

through hundreds of components.

When backend introduces:

EXAMPLE / FLOW

```
v2
```

update adapters/clients deliberately.

92. Frontend Anti-Corruption Layer

Suppose Reporting API returns:

JSON

```
{
  "campaignKey": 10,
  "raisedAmt": 250000
}
```

but UI wants:

TYPESCRIPT

```
interface CampaignPerformance {
  id: number;
  amountRaised: number;
}
```

Use mapping in your data-access layer.

Do not let backend naming infect every template.

Concept:

EXAMPLE / FLOW

```
API response
    |
    v
mapper
    |
    v
UI/domain view model
```

93. Facade Pattern in Angular

A component should not coordinate five services.

Bad:

EXAMPLE / FLOW

```
DashboardComponent
  injects:
  CampaignApi
  GiftApi
  PaymentApi
  TaskApi
  ReportingApi
```

Better:

EXAMPLE / FLOW

```
DashboardComponent
      |
      v
DashboardFacade
      |
      v
ReportingApiClient
```

The facade exposes:

EXAMPLE / FLOW

```
summary
loading
error
reload()
```

94. Adapter Pattern

Excellent for API models.

EXAMPLE / FLOW

```
GiftApiResponse
       |
       v
GiftAdapter
       |
       v
GiftViewModel
```

Also useful for:

EXAMPLE / FLOW

```
chart library models
date libraries
external component libraries
```

95. Strategy Pattern

Frontend example:

Different display strategies based on payment state:

EXAMPLE / FLOW

```
PaymentStatusPresenter
       |
       +---- Pending
       +---- Succeeded
       +---- Failed
       +---- Refunded
```

But do not create a pattern hierarchy when a simple mapping suffices.

Often:

TYPESCRIPT

```
Record<PaymentStatus, StatusPresentation>
```

is simpler.

96. Repository Pattern in Angular?

Do not blindly copy backend repositories into Angular.

Angular is not your database access layer.

Better terminology:

EXAMPLE / FLOW

```
API Client
Data Access Service
Store
Facade
```

If you call a class:

EXAMPLE / FLOW

```
GiftRepository
```

it can work conceptually, but it often creates unnecessary confusion with backend persistence repositories.

For this project:

EXAMPLE / FLOW

```
GiftApiClient
```

is clearer.

97. Dependency Injection

Angular has dependency injection.

Use:

TYPESCRIPT

```
private readonly api =
  inject(GiftApiClient);
```

instead of:

TYPESCRIPT

```
new GiftApiClient(...)
```

DI lets Angular manage:

EXAMPLE / FLOW

```
dependencies
lifetime
testing
replacement
```

providedIn root

Example:

TYPESCRIPT

```
@Injectable({
  providedIn: 'root'
})
export class CurrentUserService {
}
```

means one root application service instance.

Good for:

EXAMPLE / FLOW

```
authentication
global configuration
cross-application utilities
```

Feature-Level Providers

Feature state can be scoped to a route/page tree.

Example:

EXAMPLE / FLOW

```
ConstituentStore
```

may be created only when entering the Constituent feature.

This prevents stale state leaking across unrelated features and improves ownership.

98. Pipes

Use pipes for presentation transformations.

Examples:

EXAMPLE / FLOW

```
currency
date
percentage
status display
```

Avoid placing heavy business logic in pipes.

Currency

Gift:

EXAMPLE / FLOW

```
1000 USD
```

display:

EXAMPLE / FLOW

```
$1,000.00
```

using Angular currency formatting.

Do not permanently turn numeric values into formatted strings inside your state model.

Keep:

EXAMPLE / FLOW

```
number / decimal representation
```

separate from display.

99. Custom Directive

Useful example:

EXAMPLE / FLOW

```
appHasPermission
```

Concept:

HTML

```
<button *appHasPermission="'payments.refund'">
  Refund
</button>
```

A modern structural/helper approach can improve repeated permission UI.

Again:

EXAMPLE / FLOW

```
frontend permission
→ UI only

backend authorization
→ real security
```

100. Date Handling

Backend stores timestamps:

EXAMPLE / FLOW

```
UTC
```

Angular receives:

EXAMPLE / FLOW

```
ISO timestamp.
```

Display in the user's appropriate timezone.

Business dates such as:

EXAMPLE / FLOW

```
Campaign StartDate
GiftDate
```

may represent date-only values and should not accidentally shift because of timezone conversion.

Be deliberate about:

EXAMPLE / FLOW

```
date-only
vs
instant/timestamp.
```

101. Error Boundaries / Error Presentation

Angular does not use React-style error boundaries, but build UI-level failure isolation.

For example dashboard:

EXAMPLE / FLOW

```
Campaign chart fails
```

should not necessarily blank:

EXAMPLE / FLOW

```
entire Dashboard.
```

Each dashboard widget can have:

EXAMPLE / FLOW

```
loading
data
empty
error
```

states.

102. Offline Behavior

This is not initially an offline-first application.

Do not queue financial mutations in browser storage and replay them later without an intentional design.

Especially avoid automatically replaying:

EXAMPLE / FLOW

```
Refund
Payment
Gift creation
```

because these require strong server-side idempotency and current authorization/state.

103. Responsive Design

Users may access the application on:

EXAMPLE / FLOW

```
desktop
tablet
phone
```

Main admin workflows may be desktop-first, but critical screens should remain usable at smaller sizes.

For tables:

EXAMPLE / FLOW

```
desktop
→ table

small screen
→ card/list or horizontal strategy
```

Do not shrink a 15-column payment table into unreadable text.

104. Page Layout Standards

Each business page should have consistent:

EXAMPLE / FLOW

```
Title
Primary action
Filters
Content
Empty/error state
```

Example:

EXAMPLE / FLOW

```
Constituents                              [+ New Constituent]

Search [_____________] Status [Active ▼]

-------------------------------------------------------------
Name              Email                       Status
-------------------------------------------------------------
John Smith        john@example.com             Active
Jane Doe          jane@example.com             Active
```

105. Navigation Unsaved Changes

User edits:

EXAMPLE / FLOW

```
Gift
```

then tries to leave.

If form is dirty:

EXAMPLE / FLOW

```
You have unsaved changes.
Leave this page?
```

Use a route-deactivation guard or equivalent navigation strategy.

Do not trigger this for forms that have not changed.

106. Route Resolvers vs Modern Data Loading

Traditional Angular provides route resolvers.

They can ensure:

EXAMPLE / FLOW

```
data loaded
before route activates.
```

But this delays page navigation.

Modern Angular also provides signal/resource-oriented route data mechanisms, including route resources that can support parallel and non-blocking loading.

For this application:

EXAMPLE / FLOW

```
critical small record
→ resolver/resource may be suitable

large dashboard
→ load page immediately + skeleton
```

Do not block navigation for every API call.

107. Resource APIs

Modern Angular provides reactive resource APIs for asynchronous data.

A feature can use:

EXAMPLE / FLOW

```
route parameter
      |
      v
resource
      |
      v
API
      |
      v
loading/error/value signals
```

They can be very useful for signal-first screens.

Still learn RxJS because:

EXAMPLE / FLOW

```
MSAL
complex streams
forms
events
existing enterprise code
```

heavily use Observable concepts.

108. Dashboard State Example

Conceptually:

TYPESCRIPT

```
readonly dateRange =
  signal<DateRange>(
    currentMonth()
  );

readonly executiveSummary =
  resource({
    params: () => ({
      from: this.dateRange().from,
      to: this.dateRange().to
    }),

    loader: ({ params }) =>
      firstValueFrom(
        this.api.getExecutiveSummary(params)
      )
  });
```

This demonstrates:

EXAMPLE / FLOW

```
Signal state
Reactive data loading
Loading/error states
```

without manual state boilerplate.

109. Authentication Events

MSAL exposes authentication events.

Useful events:

EXAMPLE / FLOW

```
login success
logout
token acquisition
account changes
```

Subscribe only where needed and clean up subscriptions.

Microsoft's MSAL Angular guidance explicitly shows broadcast events and stresses unsubscribing from long-lived subscriptions.

110. MSAL Token Mapping

Configure protected resources so that requests to:

EXAMPLE / FLOW

```
Fundraising API
```

receive:

EXAMPLE / FLOW

```
fundraising scope
```

while:

EXAMPLE / FLOW

```
Reporting API
```

receives:

EXAMPLE / FLOW

```
reporting scope.
```

Do not attach every token to every external URL.

111. Authentication Failure

If API returns:

EXAMPLE / FLOW

```
401
```

possible actions:

EXAMPLE / FLOW

```
silent token acquisition
interactive reauthentication when required
```

Do not create an infinite:

EXAMPLE / FLOW

```
401
→ retry
→ 401
→ retry
```

loop.

112. 403 Handling

`403 Forbidden` means:

EXAMPLE / FLOW

```
user is authenticated
but not allowed.
```

Show:

EXAMPLE / FLOW

```
You do not have permission to perform this action.
```

Do not automatically log the user out for every 403.

113. 429 Handling

API may return:

EXAMPLE / FLOW

```
429 Too Many Requests.
```

Frontend should:

EXAMPLE / FLOW

```
respect Retry-After if present
avoid hammering
show useful feedback
```

114. Admin User Management

Remember:

EXAMPLE / FLOW

```
AppUser
```

does not store passwords.

Admin UI may manage:

EXAMPLE / FLOW

```
application role mapping
user active/inactive state
job title
application permissions
```

Actual identity/password/MFA remains:

EXAMPLE / FLOW

```
Microsoft Entra.
```

115. App Registration Is Not Angular Admin CRUD

Do not expose:

EXAMPLE / FLOW

```
Create Entra App Registration
```

as an ordinary tenant-admin button.

Cloud identity configuration belongs in:

EXAMPLE / FLOW

```
platform administration / IaC
```

unless you intentionally build a privileged identity-management product.

116. Development Environments

Local configuration:

EXAMPLE / FLOW

```
Angular:
http://localhost:4200

Fundraising API:
https://localhost:7001

Payment API:
https://localhost:7002

Reporting API:
https://localhost:7003
```

Production:

EXAMPLE / FLOW

```
Angular:
https://app.example.com

APIs:
https://api.example.com
```

Angular code should not have environment-specific logic spread everywhere.

117. Build-Time vs Runtime Configuration

Angular environment files are compile-time configuration.

For container/static deployments, runtime configuration can be useful if you want the same Angular build promoted through:

EXAMPLE / FLOW

```
Dev
Staging
Prod.
```

Pattern:

EXAMPLE / FLOW

```
Angular startup
      |
      v
/app-config.json
      |
      v
API URL
Entra IDs
Scopes
```

This avoids rebuilding the application only to change:

EXAMPLE / FLOW

```
API URL.
```

Remember:

> ★ Runtime Angular configuration still cannot contain secrets.

118. Testing Strategy

Use several layers:

EXAMPLE / FLOW

```
Unit tests
Component tests
Service/API-client tests
Guard/interceptor tests
Integration tests
E2E tests
```

Angular CLI's current default testing setup uses Vitest for new projects.

119. Unit Test

Example:

EXAMPLE / FLOW

```
allocationTotal computed signal
```

Test:

TYPESCRIPT

```
it('calculates remaining allocation', () => {
  ...
});
```

Pure calculations should not require:

EXAMPLE / FLOW

```
HTTP
DOM
MSAL
```

120. Signal Forms Tests

Test validation logic independently where possible.

Example:

EXAMPLE / FLOW

```
Gift amount = 1,000
Allocation = 900

Expected:
invalid
```

Angular's Signal Forms guidance recommends isolated tests for validation and field state, and DOM/component-bound tests when behavior depends on actual user interaction/rendering.

121. Component Test

Test:

EXAMPLE / FLOW

```
PaymentStatusBadge
```

Input:

EXAMPLE / FLOW

```
Succeeded
```

Expected DOM:

EXAMPLE / FLOW

```
Succeeded
```

and appropriate semantic styling/accessibility.

122. API Client Tests

Angular provides:

EXAMPLE / FLOW

```
HttpTestingController
```

for testing requests without hitting the real backend.

Example test:

EXAMPLE / FLOW

```
GiftApiClient.getById(10000)
```

asserts request:

EXAMPLE / FLOW

```
GET /gifts/10000
```

then supplies mock response.

123. Interceptor Tests

Verify:

EXAMPLE / FLOW

```
correlation header
error mapping
loading behavior
```

without relying on a production API.

MSAL-specific token behavior should follow Microsoft's supported library integration rather than recreating token logic yourself.

124. Guard Tests

Test:

EXAMPLE / FLOW

```
Fundraiser
→ can access /gifts
```

and:

EXAMPLE / FLOW

```
Fundraiser
→ cannot access /payments/refunds
```

Again, this tests UX navigation—not backend security.

125. E2E Tests

Use a browser automation tool such as:

EXAMPLE / FLOW

```
Playwright
```

for critical workflows.

Examples:

EXAMPLE / FLOW

```
Login
Search Constituent
Create Constituent
Create Gift
Allocate Gift
Complete Task
View Payment
Finance user refund workflow
```

Critical E2E Scenario

EXAMPLE / FLOW

```
Login as Fundraiser
      |
      v
```

```
Search John Smith
      |
      v
Open profile
      |
      v
Record $1,000 Gift
      |
      v
Allocate 700 / 300
      |
      v
Submit
      |
      v
Gift confirmation
```

This tests the frontend and backend as one business journey.

126. Different User Personas in Tests

Create test identities/claims for:

EXAMPLE / FLOW

```
Fundraiser
FinanceUser
ReportingUser
TenantAdministrator
```

Then verify role-sensitive UI.

127. Performance Testing

Frontend performance measurements should include:

EXAMPLE / FLOW

```
initial bundle
lazy chunk size
first route load
API response times
rendering large lists
chart load
memory leaks
```

Do not judge performance only from:

EXAMPLE / FLOW

```
it feels fast on my laptop.
```

Angular DevTools

Use Angular DevTools during development to inspect:

EXAMPLE / FLOW

```
component tree
performance
change detection
```

along with browser performance/network tools.

128. Bundle Budgets

Configure build budgets so the build warns/fails when bundles grow unexpectedly.

Example concern:

EXAMPLE / FLOW

```
developer imports huge charting library
into root bundle
```

and initial load jumps dramatically.

Budgets create an early warning.

129. Tree Shaking

Keep imports specific.

Avoid libraries that pull massive unused functionality where a smaller alternative fits.

Modern Angular build tooling can eliminate unused code when packages are tree-shakable.

130. Third-Party Libraries

Before adding a library ask:

EXAMPLE / FLOW

```
Is it maintained?
Angular 22 compatible?
Accessible?
Tree-shakable?
Security history?
Bundle impact?
Could Angular already do this?
```

Do not install a package for every five-line utility.

131. Frontend Logging

Do not scatter:

TYPESCRIPT

```
console.log(...)
```

through production code.

Create a logging abstraction.

Development:

EXAMPLE / FLOW

```
console
```

Production:

EXAMPLE / FLOW

```
structured telemetry
```

with care not to log sensitive data.

Never Log

EXAMPLE / FLOW

```
access token
donor sensitive information unnecessarily
payment provider secret
full payment information
password
```

132. Telemetry

Frontend telemetry can capture:

EXAMPLE / FLOW

```
page load
route navigation
API failure
unhandled error
performance metrics
```

and correlate with backend traces.

Later integrate with:

EXAMPLE / FLOW

```
Application Insights / Azure Monitor
```

as appropriate.

133. User Experience for API Failures

Different failures should have different UX.

EXAMPLE / FLOW

```
400
→ show validation problem

401
→ authentication flow

403
→ permission message

404
→ record unavailable

409
→ concurrency/business conflict

429
→ retry later

500
→ generic server failure + trace ID
```

Do not turn all status codes into the same red toast.

134. Feature Permission Matrix

Example:

Feature	Fundraiser	Manager	Finance	Reporting	Admin
Constituent View	Yes	Yes	Limited/Yes	Read	Yes
Constituent Edit	Yes	Yes	No	No	Yes
Create Gift	Yes	Yes	No/Policy	No	Yes
Campaign Manage	Limited	Yes	No	Read	Yes
Payments View	Limited	Yes	Yes	Read	Yes
Refund Payment	No	No	Yes	No	Policy
Settlements	No	No	Yes	Read	Policy
Reports	Limited	Yes	Yes	Yes	Yes
User Admin	No	No	No	No	Yes

This table is a UI design guide.

The actual policy authority remains the backend.

135. Angular Business Route Map

EXAMPLE / FLOW

```
/login

/dashboard

/constituents
/constituents/new
/constituents/:id
/constituents/:id/edit
/constituents/:id/gifts
/constituents/:id/interactions
/constituents/:id/tasks
/constituents/:id/timeline

/campaigns
/campaigns/new
/campaigns/:id
/campaigns/:id/edit
/campaigns/:id/performance
```

```
/campaigns/:id/history
```

EXAMPLE / FLOW

```
/funds
/funds/new
/funds/:id

/gifts
/gifts/new
/gifts/:id
/gifts/:id/edit

/tasks
/tasks/open
/tasks/overdue
/tasks/:id

/payments
/payments/:id
/payments/:id/refunds

/settlements
/settlements/:id
```

EXAMPLE / FLOW

```
/reports
/reports/executive
/reports/giving
/reports/campaigns
/reports/funds
/reports/payments

/admin
/admin/users
/admin/integrations
/admin/system
```

136. Route Configuration Strategy

Top-level route:

EXAMPLE / FLOW

```
/constituents
```

lazy-loads the Constituent feature.

Then feature routes:

TYPESCRIPT

```
export const CONSTITUENT_ROUTES: Routes = [
  {
    path: '',
    loadComponent: () =>
      import('./pages/constituent-list/constituent-list')
        .then(m => m.ConstituentListPage)
  },

  {
    path: 'new',
    loadComponent: () =>
      import('./pages/constituent-create/constituent-create')
        .then(m => m.ConstituentCreatePage)
  },

  {
    path: ':id',
    loadComponent: () =>
      import('./pages/constituent-details/constituent-details')
        .then(m => m.ConstituentDetailsPage)
```

TYPESCRIPT

```
  }
];
```

Each page remains standalone.

137. Route Titles

Define meaningful page titles.

Example:

EXAMPLE / FLOW

```
Constituents – Fundraising Platform

John Smith – Fundraising Platform

Payments – Fundraising Platform
```

This improves:

EXAMPLE / FLOW

```
accessibility
browser tabs
history
```

138. Breadcrumbs

Use route metadata rather than manually coding breadcrumbs on every page.

Example:

EXAMPLE / FLOW

```
Constituents
>
John Smith
>
Gifts
```

139. Unsaved Changes Guard

Form pages:

EXAMPLE / FLOW

```
Create Constituent
Edit Campaign
Edit Gift
```

should protect users from accidental navigation.

But after successful save:

EXAMPLE / FLOW

```
mark form clean
```

so normal navigation does not continue prompting.

140. Dashboard Facade

Example architecture:

EXAMPLE / FLOW

```
DashboardPage
      |
      v
DashboardFacade
      |
      v
ReportingApiClient
      |
      v
Reporting API
```

Facade state:

EXAMPLE / FLOW

```
summary
monthlyGiving
campaignPerformance
paymentPerformance
loading
errors
```

141. Constituents Architecture

EXAMPLE / FLOW

```
ConstituentListPage
      |
      v
ConstituentStore
      |
      v
ConstituentApiClient
      |
      v
Fundraising API
```

142. Payment Architecture

EXAMPLE / FLOW

```
PaymentDetailPage
      |
```

```
        v
PaymentFacade
        |
        +---- PaymentApiClient
        |
        +---- CurrentUserService
        |
        +---- DialogService
        |
        v
Payment API
```

143. Reporting Architecture

EXAMPLE / FLOW

```
CampaignReportPage
        |
        v
CampaignReportFacade
        |
        v
ReportingApiClient
        |
        v
Reporting API
        |
        v
ReportingDb
```

No direct SQL from Angular.

Ever.

144. Frontend Does Not Know Databases

Angular should know:

EXAMPLE / FLOW

```
HTTP API contracts.
```

It should **not** know:

EXAMPLE / FLOW

```
FundraisingDb connection
PaymentDb schema
stored procedure name
table name
```

For example Angular knows:

EXAMPLE / FLOW

```
POST /gifts
```

not:

EXAMPLE / FLOW

```
fundraising.usp_CreateGift.
```

This is an important separation.

145. Frontend Does Not Talk Directly to Service Bus

Angular should not publish:

EXAMPLE / FLOW

```
GiftCreated
```

directly to Azure Service Bus.

Correct:

EXAMPLE / FLOW

```
Angular
   |
   v
Fundraising API
   |
   v
SQL + Outbox
   |
   v
Worker
   |
   v
Service Bus
```

This protects:

EXAMPLE / FLOW

```
business integrity
credentials
authorization
event consistency.
```

146. Frontend Does Not Use Key Vault

Angular runs in the user's browser.

It should not receive Key Vault credentials.

Key Vault belongs to:

EXAMPLE / FLOW

```
backend workloads.
```

147. Angular + APIM

Production request:

EXAMPLE / FLOW

```
Angular
      |
      v
https://api.example.com
      |
      v
Azure API Management
      |
      +---- Fundraising API
      |
      +---- Payment API
      |
      +---- Reporting API
```

Angular does not need to know Container App internal addresses.

148. Angular + Front Door

Final user path:

EXAMPLE / FLOW

```
Browser
   |
   v
Azure Front Door
   |
   +---- Angular static origin
   |
   +---- /api
            |
            v
          APIM
```

This can give a unified public application domain architecture.

149. Azure Deployment

Build:

EXAMPLE / FLOW

```
npm ci

ng test --watch=false

ng build
```

Then deploy the production build.

Possible target:

EXAMPLE / FLOW

```
Azure Static Web Apps
```

or:

EXAMPLE / FLOW

```
static hosting + Front Door.
```

The Azure document already established the deployment choices.

150. SPA Navigation Fallback

Because Angular routes exist client-side:

EXAMPLE / FLOW

```
/constituents/1001
```

the hosting platform must route unknown SPA paths to:

EXAMPLE / FLOW

```
index.html
```

rather than returning server 404.

Azure Static Web Apps supports SPA navigation fallback configuration.

151. GitHub Actions — Angular CI

Conceptual pipeline:

EXAMPLE / FLOW

```
Pull Request
      |
      v
npm ci
      |
      v
lint
      |
      v
unit tests
      |
      v
production build
```

Only merge after successful checks.

Angular CD

EXAMPLE / FLOW

```
main branch
      |
      v
build Angular artifact
      |
      v
deploy Dev
```

```
      |
      v
E2E / smoke test
      |
      v
Staging
      |
      v
approval
      |
      v
Production
```

152. Runtime Configuration Promotion

A strong deployment model is:

EXAMPLE / FLOW

```
same Angular build artifact
      |
      +---- Dev runtime config
      |
      +---- Staging runtime config
      |
      +---- Prod runtime config
```

If your chosen hosting/build strategy supports it cleanly.

This mirrors the backend:

EXAMPLE / FLOW

```
build once
promote same artifact.
```

153. Angular Testing Pyramid

EXAMPLE / FLOW

```
            E2E
          /     \
         /       \
    Integration
      /       \
     /         \
Component      API-client
      \         /
       \       /
       Unit tests
```

Most tests should be:

EXAMPLE / FLOW

```
fast unit/component tests
```

with fewer expensive full E2E tests.

154. What to Unit Test

Good:

EXAMPLE / FLOW

```
validators
computed state
mappers
permission rules
API adapters
stores
facades
utility functions
```

What Not to Unit Test

Do not test:

EXAMPLE / FLOW

```
that Angular's @if works
that Array.map works
that HttpClient exists
```

Test your logic.

155. Complete Angular Topic Map

Angular Topic	Project Use
Standalone components	all new UI
Components	every screen/widget
Inputs	reusable display components
Outputs	child-to-parent events
Signals	UI/feature state
computed	totals/derived UI
effect	limited side effects
RxJS	HTTP/events/search
Observable	HTTP/MSAL streams
switchMap	searches
concatMap	ordered async workflows
mergeMap	controlled independent work
exhaustMap	duplicate-submit protection
takeUntilDestroyed	subscription cleanup
DI	services/facades/API clients
Router	navigation
Lazy loading	all major features
Preloading	likely-used features
@defer	expensive page sections
Guards	auth/role/navigation
Resolver/resource	route data where appropriate
HttpClient	backend calls
Interceptor	auth/errors/correlation
Signal Forms	new project forms
Reactive Forms	enterprise/legacy knowledge
Validation	forms

Angular Topic	Project Use
FormArray	allocations
Pipes	presentation
Directives	reusable UI/permissions
Facade	page/business coordination
Adapter	API → UI mapping
Store	feature state
Pagination	all large lists
Filtering	business searches
Sorting	tables
Virtual scroll	selected large UI lists
MSAL	Entra authentication
Access tokens	API calls
Roles	menus/actions
ProblemDetails	error handling
Vitest	unit tests
HttpTestingController	API client tests
Playwright concept	E2E
Accessibility	entire UI
Zoneless	default modern execution
OnPush	default modern CD
Runtime config	environment promotion
Static Web Apps	Azure hosting option

156. Full Business UI Map

EXAMPLE / FLOW

```
                                ANGULAR
                                   |
```

```
       +--------------------+--------------------+
       |                    |                    |
       v                    v                    v
  Fundraising           Payments             Reporting
       |                    |                    |
       |                    |                    |
 Constituents          Payments           Executive
 Campaigns             Attempts           Campaign
 Funds                 Refunds            Giving
 Gifts                 Settlements        Funds
 Interactions                             Payments
 Tasks                                    Engagement
```

Administration surrounds these:

EXAMPLE / FLOW

```
Users
Roles
Integration status
Application health
```

157. Full Authentication Architecture

EXAMPLE / FLOW

```
Browser
  |
  v
Angular
  |
  v
MSAL
  |
  v
Microsoft Entra ID
  |
  | Authorization Code + PKCE
  v
Access Token
  |
  v
Angular HTTP
  |
  v
Front Door
```

EXAMPLE / FLOW

```
  |
  v
API Management
  |
  v
ASP.NET Core
  |
  v
Authorization Policy
```

PKCE is required for SPA authorization-code flows, and Microsoft's identity platform recommends this flow rather than implicit grant.

158. Full Gift UI Flow

EXAMPLE / FLOW

```
Fundraiser
     |
     v
Constituent Profile
     |
     v
Record Gift
     |
     v
Gift Signal Form
     |
     +---- Constituent
     +---- Campaign
     +---- Amount
     +---- Allocations
     |
     v
Frontend Validation
     |
     v
```

EXAMPLE / FLOW

```
POST /gifts
     |
     v
API
     |
     v
```

```
SQL Transaction
      |
      +---- Gift
      +---- GiftAllocation
      +---- Outbox
      |
      v
201 Created
      |
      v
Angular Gift Detail
```

Later asynchronously:

EXAMPLE / FLOW

```
GiftCreated
    |
    v
Payment Service
```

Angular can update payment status when refreshed/polled/through a future real-time mechanism.

159. Payment UI Flow

EXAMPLE / FLOW

```
Finance User
      |
      v
Payment Search
      |
      v
Payment Detail
      |
      +---- Attempts
      +---- Refunds
      +---- Settlement
      |
      v
Request Refund
      |
      v
Confirmation Dialog
      |
      v
POST /refunds
```

EXAMPLE / FLOW

```
    |
    v
Payment API
```

160. Dashboard Flow

EXAMPLE / FLOW

```
User
 |
 v
Dashboard
 |
 v
ReportingFacade
 |
 v
Reporting API
 |
 v
ReportingDb
 |
 v
Aggregated response
 |
 v
Signals
 |
```

EXAMPLE / FLOW

```
 +---- Summary cards
 +---- charts
 +---- alerts
```

This keeps heavy analytical work out of the browser.

161. Error Handling Architecture

EXAMPLE / FLOW

```
HTTP Error
    |
```

```
    v
Interceptor
    |
    +---- 401 → auth
    |
    +---- 403 → permission
    |
    +---- 429 → throttling
    |
    +---- 500 → generic/global
    |
    v
Feature handler
    |
    +---- validation
    +---- concurrency
    +---- business error
    |
```

EXAMPLE / FLOW

```
    v
User-friendly UI
```

162. State Architecture

EXAMPLE / FLOW

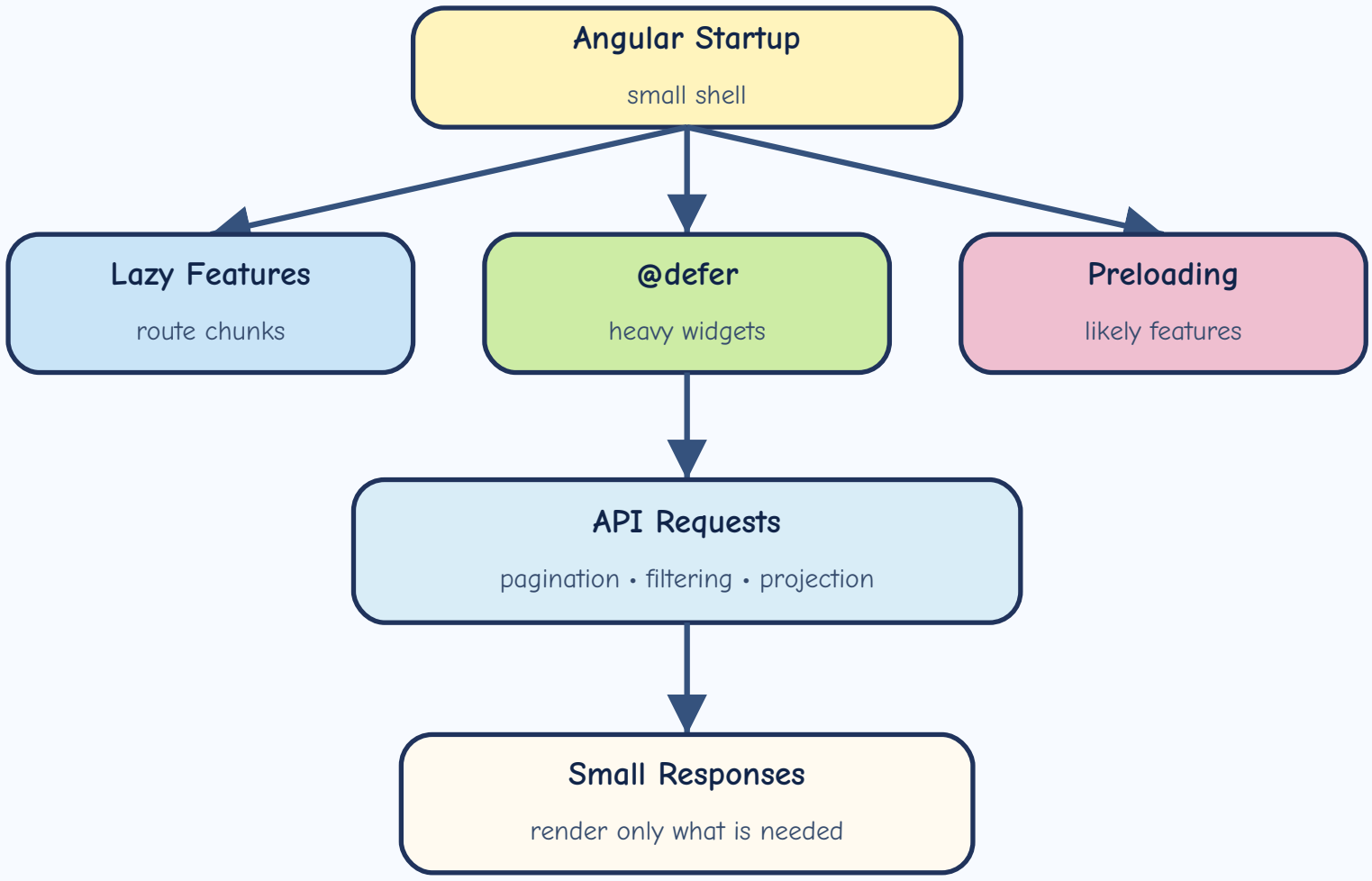
```
SERVER STATE
API / SQL
     |
     v
API Client
     |
     v
Feature Store / Facade
     |
     +---- signals
     +---- RxJS
     |
     v
Components
```

Do not duplicate entire backend databases in frontend state.

Frontend state should contain only what the current user experience needs.

DIAGRAM

Angular Performance - Load Less, Later



Keep the startup shell small, lazy-load features, defer non-critical widgets, and ask the server for only the rows and fields the screen needs.

163. Performance Architecture

EXAMPLE / FLOW

```
Angular startup
      |
      +---- small shell
      |
      +---- lazy features
      |
      +---- defer heavy widgets
      |
      v
API
      |
      +---- pagination
      +---- filtering
      +---- projection
      |
      v
small responses
```

The major performance rule is:

> ★ **Do less work, load less data, and load it only when needed.**

164. Recommended Build Sequence

Do not build every feature simultaneously.

Stage 1 — Angular Fundamentals

Learn through project examples:

EXAMPLE / FLOW

```
TypeScript
components
templates
bindings
@if
@for
inputs
outputs
services
DI
signals
computed
RxJS basics
```

Stage 2 — Application Shell

Build:

EXAMPLE / FLOW

```
layout
sidebar
header
router outlet
navigation
404 page
```

Stage 3 — Routing

Add:

EXAMPLE / FLOW

```
feature routes
lazy loading
page titles
breadcrumbs
```

Stage 4 — API Infrastructure

Build:

EXAMPLE / FLOW

```
HttpClient config
API configuration
ProblemDetails
error interceptor
correlation ID handling
loading patterns
```

Stage 5 — Constituents

Build completely:

EXAMPLE / FLOW

```
List
Search
Pagination
Create
Edit
Details
Addresses
Timeline
```

This is your first complete frontend CRUD feature.

Stage 6 — Campaigns and Funds

Build:

EXAMPLE / FLOW

```
Campaign CRUD
Fund CRUD
performance views
temporal history UI
```

Stage 7 — Gift Workflow

Build:

EXAMPLE / FLOW

```
Gift list
Gift form
Signal Forms
Gift allocations
cross-field validation
Gift detail
RowVersion handling
```

This is the most important business form.

Stage 8 — Tasks and Interactions

Build:

EXAMPLE / FLOW

```
Task screens
Open/overdue filters
Interaction history
Constituent timeline
```

Stage 9 — Entra / MSAL

Now add:

EXAMPLE / FLOW

```
Entra app registration integration
MSAL
login
logout
route guards
access tokens
roles
permission service
```

You can introduce authentication earlier in real production development; this learning order simply allows you to understand normal Angular/API behavior before debugging identity at the same time.

Stage 10 — Payments

Build:

EXAMPLE / FLOW

```
Payment list
Payment detail
Attempts
Refund UI
Settlement UI
Finance role guards
```

Stage 11 — Reporting

Build:

EXAMPLE / FLOW

```
Executive dashboard
Campaign charts
Giving trends
Payment analytics
filters
date ranges
```

Stage 12 — Admin

Build:

EXAMPLE / FLOW

```
User management
Role-sensitive navigation
integration status
failure dashboards
```

Stage 13 — Performance

Add/refine:

EXAMPLE / FLOW

```
lazy loading
@defer
preloading
server pagination
search debounce
bundle budgets
virtualization where appropriate
runtime configuration
```

Stage 14 — Testing

Complete:

EXAMPLE / FLOW

```
Vitest
component tests
form tests
HTTP tests
guards
interceptors
facades
E2E tests
```

Stage 15 — Azure Deployment

Connect:

EXAMPLE / FLOW

```
Angular
    |
    v
Static hosting
    |
    v
Front Door
    |
    v
APIM
    |
    v
Container Apps
```

Add GitHub CI/CD.

165. Angular Completion Checklist

Before moving to AI, you should be able to demonstrate:

EXAMPLE / FLOW

```
✓ Angular 22 project
✓ TypeScript strict mode
✓ standalone architecture
✓ feature-based folders
✓ core/shared/features separation
✓ components
✓ templates
✓ property binding
✓ event binding
✓ two-way form binding concepts
✓ @if
✓ @for
✓ @switch
✓ inputs
✓ outputs
✓ services
✓ Angular DI
✓ signals
✓ writable signals
✓ readonly signals
```

EXAMPLE / FLOW

```
✓ computed
✓ effect
✓ RxJS Observables
```

- ✓ map
- ✓ filter
- ✓ tap
- ✓ switchMap
- ✓ concatMap
- ✓ mergeMap
- ✓ exhaustMap
- ✓ debounceTime
- ✓ distinctUntilChanged
- ✓ catchError
- ✓ finalize
- ✓ combineLatest
- ✓ forkJoin
- ✓ takeUntilDestroyed
- ✓ router
- ✓ child routes
- ✓ route parameters

EXAMPLE / FLOW

- ✓ query parameters
- ✓ lazy loading
- ✓ preloading
- ✓ route guards
- ✓ unsaved-change guard
- ✓ route data
- ✓ @defer
- ✓ HttpClient
- ✓ API clients
- ✓ functional interceptors
- ✓ error interceptor
- ✓ correlation IDs
- ✓ server-side pagination
- ✓ cursor pagination concepts
- ✓ filtering
- ✓ sorting
- ✓ debounced search
- ✓ Signal Forms
- ✓ Reactive Forms
- ✓ typed forms

EXAMPLE / FLOW

- ✓ validation
- ✓ cross-field validation
- ✓ dynamic allocations
- ✓ server validation errors
- ✓ ProblemDetails
- ✓ optimistic concurrency UI
- ✓ loading states

✓ skeletons
✓ empty states
✓ error states
✓ dialogs
✓ notifications
✓ reusable UI components
✓ accessibility
✓ responsive design
✓ Angular Material/UI library concepts
✓ authentication
✓ Microsoft Entra ID
✓ MSAL
✓ OAuth 2.0

EXAMPLE / FLOW

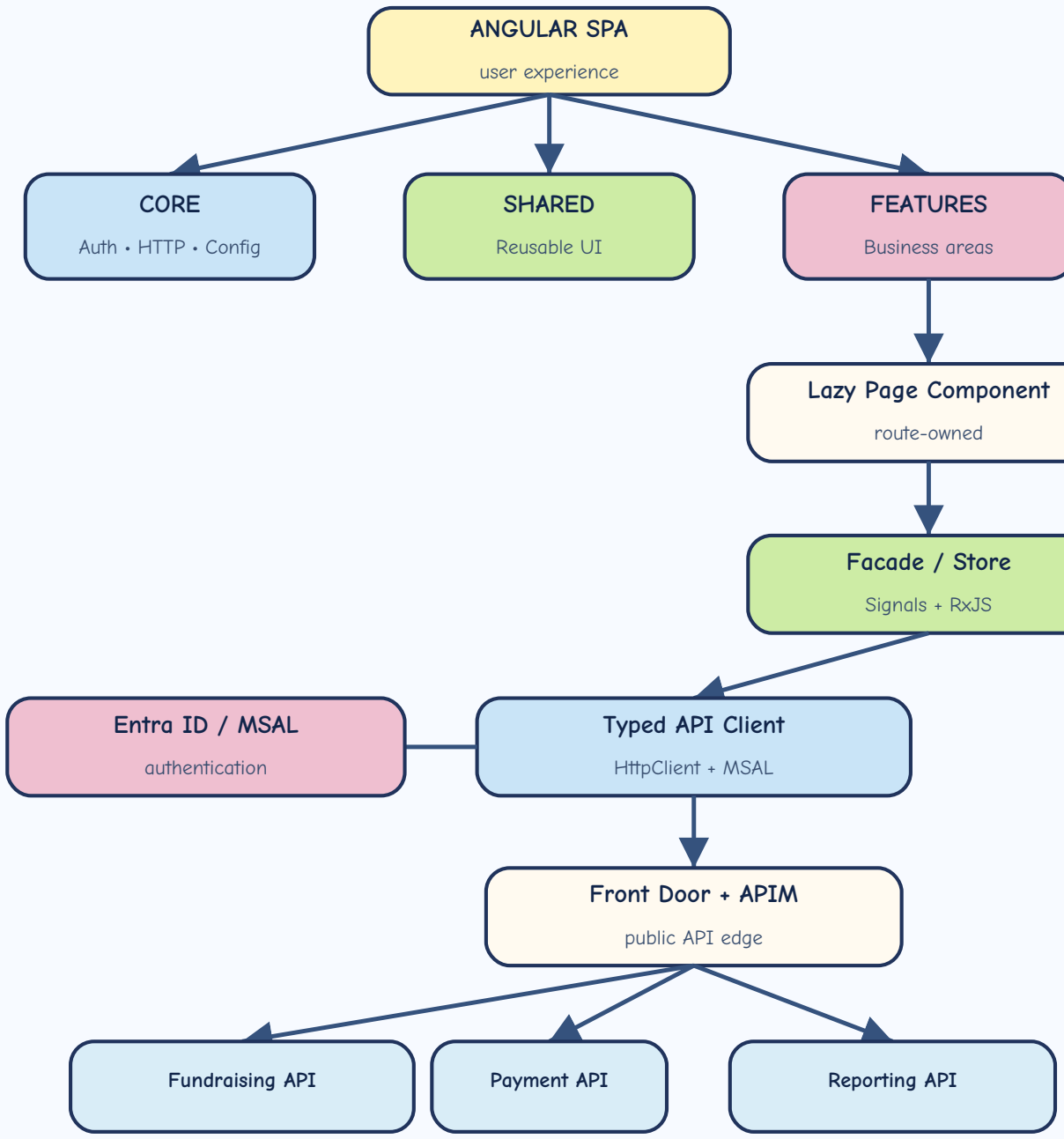
✓ Authorization Code + PKCE
✓ access tokens
✓ MSAL guard
✓ role guard
✓ scopes
✓ roles
✓ permission-based menus
✓ tenant-aware UI
✓ frontend security limitations
✓ XSS awareness
✓ CORS understanding
✓ state-management strategy
✓ feature stores
✓ facades
✓ adapters
✓ optional NgRx knowledge
✓ charts
✓ dashboard
✓ frontend performance
✓ zoneless Angular

EXAMPLE / FLOW

✓ OnPush behavior
✓ bundle optimization
✓ Vitest
✓ form tests
✓ component tests
✓ HttpTestingController
✓ guards/interceptor tests
✓ Playwright E2E concepts
✓ environment configuration
✓ runtime configuration
✓ Azure deployment
✓ GitHub CI/CD

DIAGRAM

Final Angular 22 Architecture



The frontend is feature-oriented and API-driven. MSAL handles authentication; typed data access layers call APIs through the Azure edge; backend services remain the business and security authority.

166. Final Angular Architecture

The final frontend architecture should look like:

EXAMPLE / FLOW

```
                          ANGULAR SPA
                              |
      +-----------------------+-----------------------+
      |                       |                       |
      v                       v                       v
    CORE                   SHARED                 FEATURES
      |                       |                       |
Auth/MSAL               UI Components      Constituents
```

```
Config                    Pipes              Campaigns
HTTP                      Directives         Funds
Errors                                       Gifts
Guards                                       Tasks
Current User                                 Payments
                                             Reports
                                             Admin
                          |
                          v
                       ROUTER
                          |
                 Lazy-loaded Features
```

EXAMPLE / FLOW

```
                          |
                          v
                   Page Components
                          |
                          v
                  Facade / Store
                   /          \
                  /            \
           Signals/RxJS      View State
                |
                v
             API Client
                |
                v
          HttpClient
                |
                v
         MSAL Token Layer
                |
                v
```

EXAMPLE / FLOW

```
              Front Door
                  |
                  v
           API Management
                  |
        +---------+----------+
        |         |          |
        v         v          v
  Fundraising   Payment   Reporting
     API          API       API
```

Authentication surrounds the application:

EXAMPLE / FLOW

```
Angular
   |
   v
MSAL
   |
   v
Microsoft Entra ID
```

Authorization controls:

EXAMPLE / FLOW

```
Routes
Menus
Buttons
Actions
```

while actual security remains:

EXAMPLE / FLOW

```
ASP.NET Core policies
+
tenant validation
+
SQL security.
```

167. Final Project Memory Rule

When building Angular, remember:

EXAMPLE / FLOW

```
Component
→ UI
```

EXAMPLE / FLOW

```
Signal
→ local/reactive state
```

EXAMPLE / FLOW

```
RxJS
→ async/event stream
```

EXAMPLE / FLOW

`Facade / Store`
`→ feature coordination/state`

EXAMPLE / FLOW

`API Client`
`→ HTTP communication`

EXAMPLE / FLOW

`MSAL`
`→ authentication/token acquisition`

EXAMPLE / FLOW

`Guard`
`→ navigation UX`

EXAMPLE / FLOW

`ASP.NET Authorization`
`→ actual security`

EXAMPLE / FLOW

`Signal Forms`
`→ new form state and validation`

EXAMPLE / FLOW

`Lazy Loading`
`→ feature bundle performance`

EXAMPLE / FLOW

`@defer`
`→ non-critical content performance`

EXAMPLE / FLOW

`Server Pagination`
`→ large-data scalability`

EXAMPLE / FLOW

```
Reporting API
→ dashboards and analytics
```

EXAMPLE / FLOW

```
Front Door + APIM
→ production API entry
```

168. End-to-End Frontend Journey

The entire application journey becomes:

EXAMPLE / FLOW

```
USER
 |
 v
ANGULAR
 |
 v
MSAL
 |
 v
MICROSOFT ENTRA ID
 |
 v
AUTHENTICATED ANGULAR SESSION
 |
 +-------------------------------+
 |                               |
 v                               v
FUNDRAISING UI                  FINANCE UI
 |                               |
 v                               v
```

EXAMPLE / FLOW

```
Constituents                    Payments
Campaigns                       Refunds
Funds                           Settlements
Gifts
Tasks
 |
 +-------------------------------+
                 |
                 v
```

```
                REPORTING
                   |
                Dashboard
                   |
                   v
                HttpClient
                   |
                   v
              Front Door
                   |
                   v
```

EXAMPLE / FLOW

```
          API Management
                   |
     +-----------+------------+
     |           |            |
     v           v            v
Fundraising   Payments     Reporting
   API           API          API
```

The frontend should remain:

EXAMPLE / FLOW

- strongly typed
- feature-oriented
- lazy loaded
- signal-first
- RxJS-capable
- accessible
- role-aware
- API-driven
- secure by design
- testable
- observable
- deployable independently

The core rule for this Angular project is:

> **Angular owns the user experience, not the business source of truth. Use Signals for UI and feature state, RxJS for asynchronous streams, feature facades/stores for coordination, typed API clients for HTTP, Signal Forms for new forms, server-side pagination/filtering for large data, MSAL + Entra ID for authentication, route/permission checks for UX, ASP.NET policies for real authorization, and lazy loading/deferred loading to keep the application fast. Never let the browser directly own database integrity, payment integrity, Service Bus publishing, secrets, or tenant security.**

And the performance/security memory rule is:

EXAMPLE / FLOW

```
Lazy features
+
small API responses
+
server pagination
+
signals
+
controlled RxJS
+
no unnecessary subscriptions
+
MSAL
+
role-aware UI
+
backend authorization
+
no browser secrets
+
```

EXAMPLE / FLOW

```
accessible reusable UI
+
tests
```

That completes the frontend layer of the same project:

EXAMPLE / FLOW

```
SQL Server
      ↓
.NET / Microservices
      ↓
Azure / DevOps
      ↓
Angular
```

The remaining major document is the **AI layer**: LLM fundamentals, Azure/OpenAI integration, RAG, embeddings/vector search, tool calling, agents, MCP server/client, agent security, prompt injection protection, Copilot usage, self-service reporting, fundraising assistant, payment-operations assistant, event-driven AI automation, human approval, evaluation, observability, and how all of those capabilities connect to this exact platform.